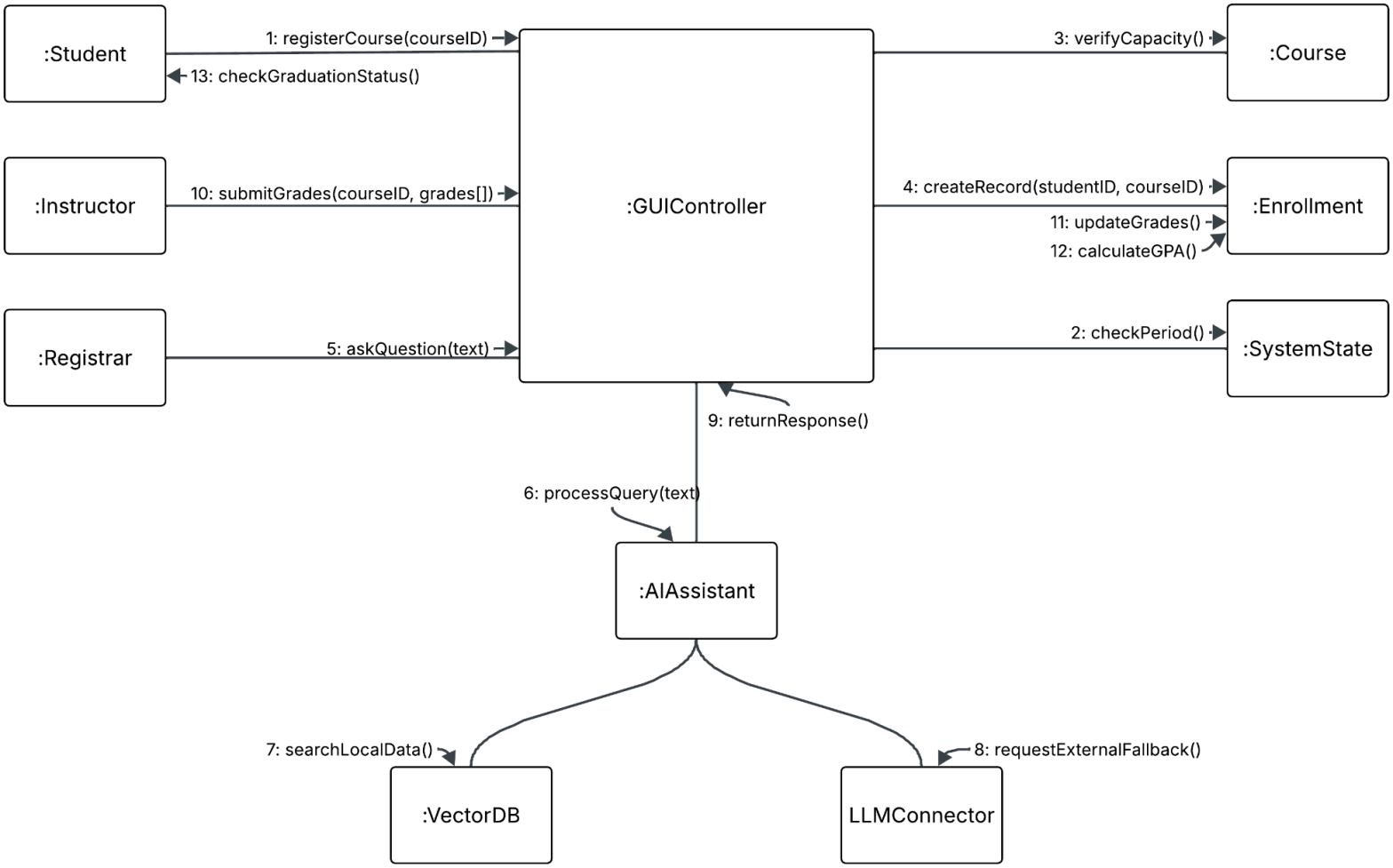
Phase II: Design Report

1. Class Collaboration Diagram



1.1 Overview

The system architecture for College0 is designed around a **Mediator Pattern** utilizing a central: GUIController. This approach ensures that the application adheres to the "Single Window" constraint mandated by the project specifications, where all user interactions are processed within a unified interface. While the E-R diagram provides a static view of the data structure, this Collaboration (Communication) Diagram illustrates the dynamic behavior of the system, showing how objects exchange messages to fulfill the academic and AI-assisted requirements.

1.2 Core System Objects

The diagram consists of several key objects that interact to execute system logic:

- **Actors (:Student, :Instructor, :Registrar):** The primary users of the system.
- **Controller (:GUIController):** The central hub that receives user input, validates state, and updates the view.
- **Academic Entities (:Course, :Enrollment):** Objects responsible for maintaining class capacities and academic records.
- **System State (:SystemState):** A global monitor that tracks the current semester period (Setup, Registration, Running, Grading).
- **AI Module (:AIAssistant, :VectorDB, :LLMConnector):** The intelligence layer that handles natural language queries.

1.3 Interaction Analysis & Logic Flow

The diagram highlights three critical communication paths derived from the system's pseudocode:

A. Course Registration Logic (Messages 1–4)

As shown in the diagram, a registration request from a :Student (1) is not passed directly to the database. Instead, the :GUIController first queries the :SystemState (2) to verify that the semester is in the "Registration" period. Only after this validation does it check the :Course capacity (3) and finally instruct the :Enrollment object (4) to create a record. This ensures strict adherence to the semester lifecycle rules.

B. AI Grounding and Fallback (Messages 5–8)

To satisfy the requirement for localized AI, the :AIAssistant follows a prioritized retrieval strategy. When a query is received (5), the assistant first attempts a local search via the :VectorDB (6). If no relevant "special" college information is found, the system communicates with the external :LLMConnector (7) to provide a general answer, which is subsequently returned to the user with a mandatory hallucination warning (8).

C. Administrative and Academic Updates (Messages 9–13)

The architecture supports complex administrative tasks, such as the :Registrar advancing the semester period (9) and :Instructors submitting grades (10). These actions trigger downstream logic in the :Enrollment and :Student objects to automate GPA calculations (11), verify graduation eligibility (12), and issue automated warnings (13) based on the criteria defined in the Phase I SRS.

1.4 Design Justification

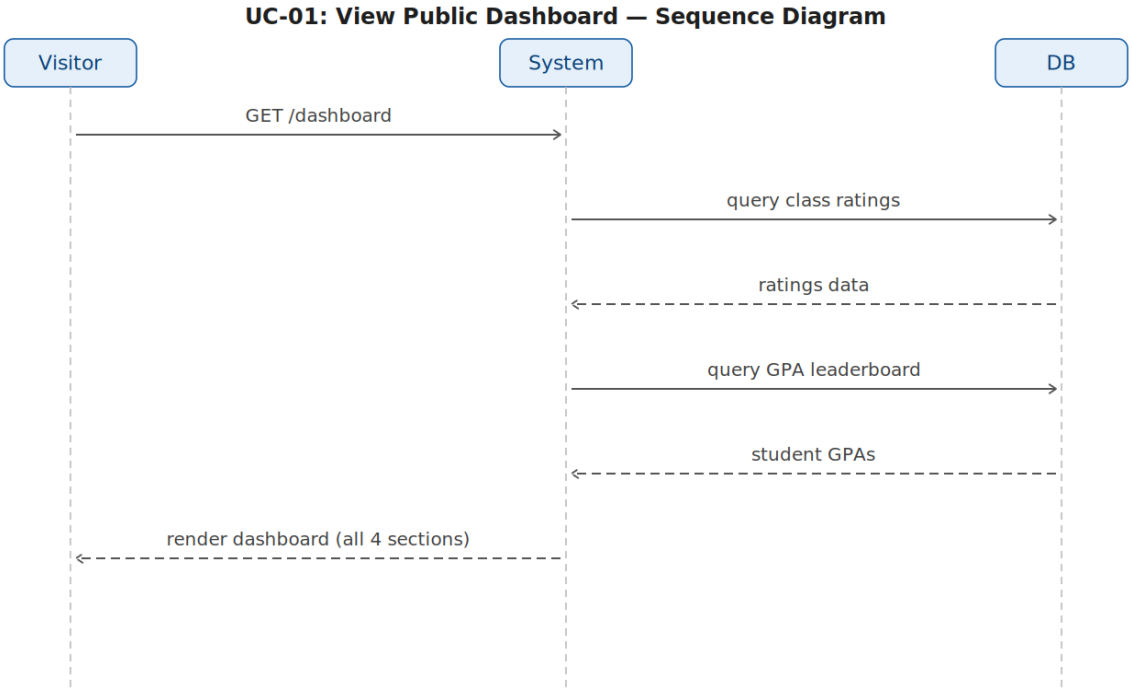
- **Single Window Consistency:** By routing all actor messages through the :GUIController, the system maintains a persistent frame, swapping internal panels rather than opening multiple windows.

- **State-Dependent Security:** The direct link between the controller and :SystemState acts as a global guardrail, preventing illegal actions (e.g., registering during the "Grading" period).
- **Data Integrity:** Intermediary communication through :Enrollment ensures that student records are updated only after multi-factor validation (prerequisites, capacity, and standing).

2. Use case Scenarios and diagrams

*Note: Normal and exceptional scenarios are counted separately.
*Petri-net done for UC-06, UC-08, and UC-13.

UC-01: View Public Dashboard



Normal Scenario

<u>A visitor navigates to the home page without logging in.</u>

1. The user opens a browser and navigates to the College0 home page.
2. The system detects no authentication is required for this page.
3. The system renders the public dashboard with four sections: a program introduction, the highest-rated classes ranked by average star rating, the lowest-rated classes, and a GPA leaderboard of students.
4. The user reads and scrolls through all four sections freely.

5. Student identities on the GPA leaderboard are displayed to the user.

Exceptional Scenarios

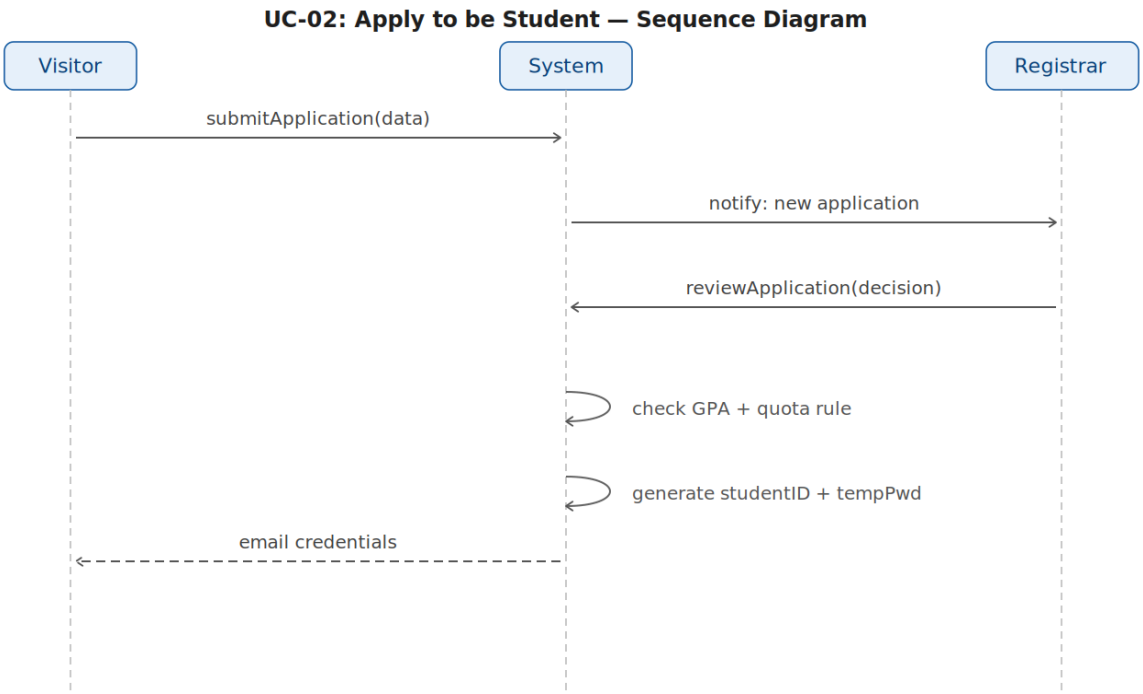
<u>No Reviews Yet (Empty Ratings)</u>

1. The user navigates to the public dashboard.
2. No students have submitted any reviews yet.
3. The highest-rated and lowest-rated class sections display empty states or placeholder text indicating no ratings are available.
4. The GPA leaderboard still displays enrolled students.

<u>No Students with GPA Data</u>

5. No students have completed any grading period yet.
6. The GPA leaderboard section renders with an empty or placeholder state.
7. The rest of the dashboard still displays normally.

UC-02: Apply to be Student



Normal Scenario

<u>A visitor with a GPA above 3.0 submits a student application and is accepted by the Registrar within quota.</u>

6. The Visitor navigates to the application form and selects 'Apply as Student.'

7. The Visitor fills in personal information and academic background, including a GPA above 3.0, and submits the form.

8. The system records the application as pending and notifies the Registrar.

9. The Registrar reviews the application. Since GPA > 3.0 and the program quota is not exceeded, the system allows acceptance without requiring justification.

10. The Registrar approves the application.

11. The system generates a unique student ID and temporary password, sets first_login = true, and emails credentials to the applicant.

12. The applicant is now a Student with active login credentials.

Exceptional Scenarios

<u>Accepting an Applicant Below 3.0 GPA or Over Quota</u>

8. The Registrar attempts to accept an applicant whose GPA is ≤ 3.0, or whose acceptance would exceed the program quota.

9. The system blocks the decision and prompts the Registrar to enter a written justification.

10. The Registrar enters the justification and resubmits.

11. The system logs the override and accepts the applicant, generating credentials as normal.

<u>Rejecting an Applicant Above 3.0 Within Quota</u>

12. The Registrar attempts to reject an applicant whose GPA > 3.0 and where quota space is available.

13. The system blocks the rejection and requires a written justification.

14. The Registrar enters the justification and confirms the rejection.

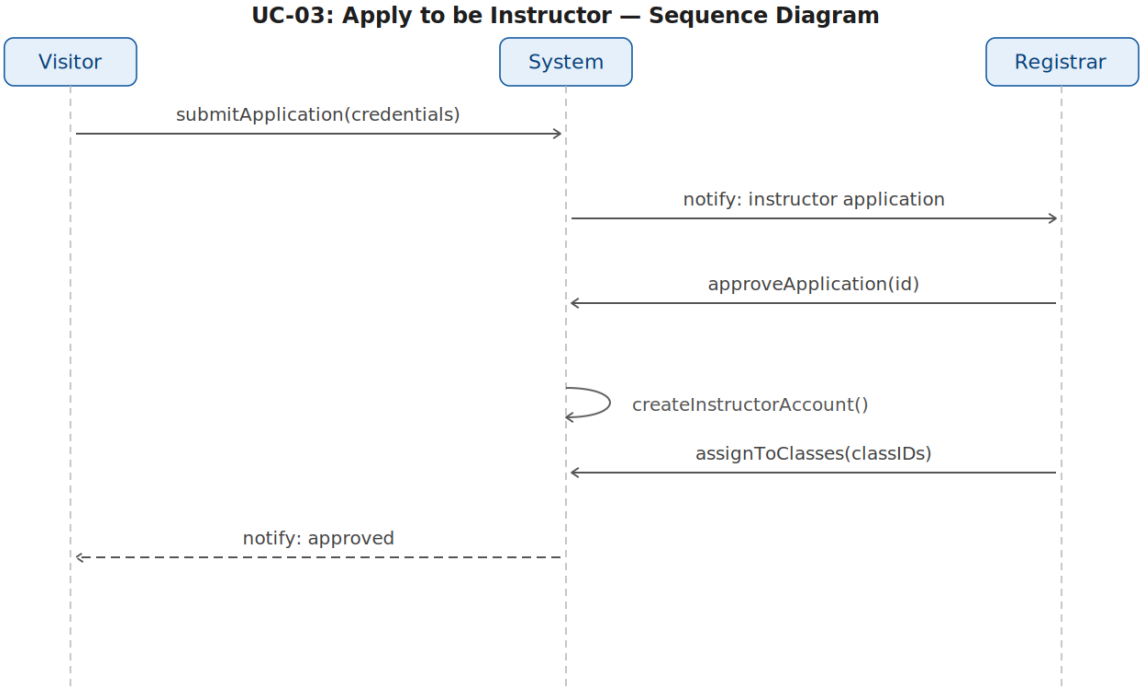
15. The system notifies the applicant of the rejection and logs the override.

<u>Applicant Rejected (Standard)</u>

16. The Registrar rejects an applicant who does not meet the GPA threshold and no override is needed.

17. The system notifies the applicant of the rejection with no further explanation required.

UC-03: Apply to be Instructor



Normal Scenario

<u>A visitor submits an instructor application and is approved by the Registrar.</u>

13. The Visitor navigates to the application form and selects 'Apply as Instructor.'
14. The Visitor fills in professional credentials and teaching background and submits.
15. The system records the application and notifies the Registrar.
16. The Registrar reviews the application and approves it.
17. The system creates an instructor account.
18. The Registrar assigns the new instructor to one or more classes.
19. The applicant receives notification and gains access to the instructor dashboard.

Exceptional Scenarios

<u>Instructor Application Rejected</u>

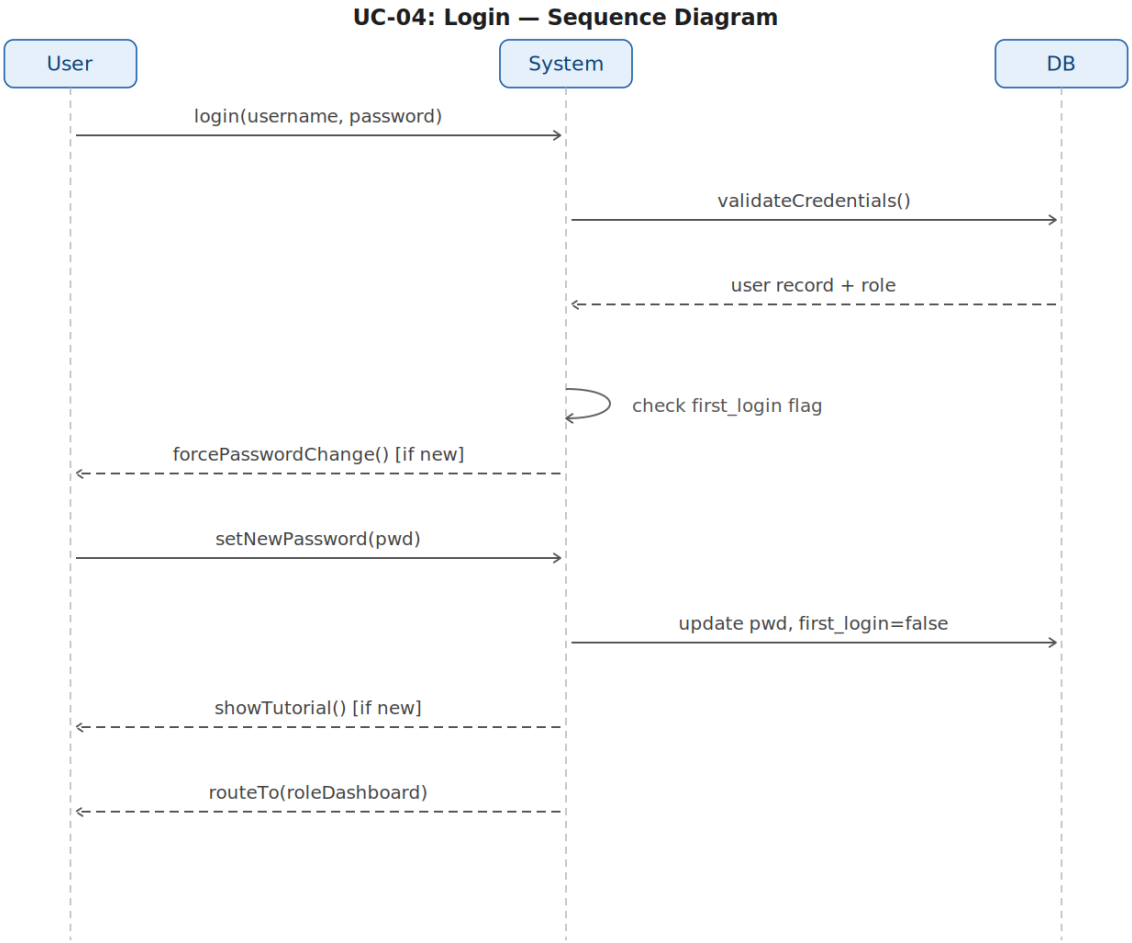
18. The Registrar reviews the instructor application and determines the applicant is not qualified.
19. The Registrar rejects the application. No written justification is required.
20. The system notifies the applicant of the rejection.
21. No account is created.

<u>Visitor Already Has an Instructor Account</u>

22. A Visitor who already has an instructor account attempts to submit another application.
23. The system detects the duplicate and rejects the submission with an error message.

24. No new application is created.

UC-04: Login



Normal Scenario

<u>A returning student (not first login) logs in successfully.</u>

20. The user navigates to the login page.
21. The user enters their username and password and clicks Login.
22. The system validates the credentials against stored records.
23. The system identifies the user's role (Student, Instructor, or Registrar).
24. The system routes the user to their role-specific home page with all permitted features available.

Exceptional Scenarios

<u>First Login — Student Must Change Password</u>

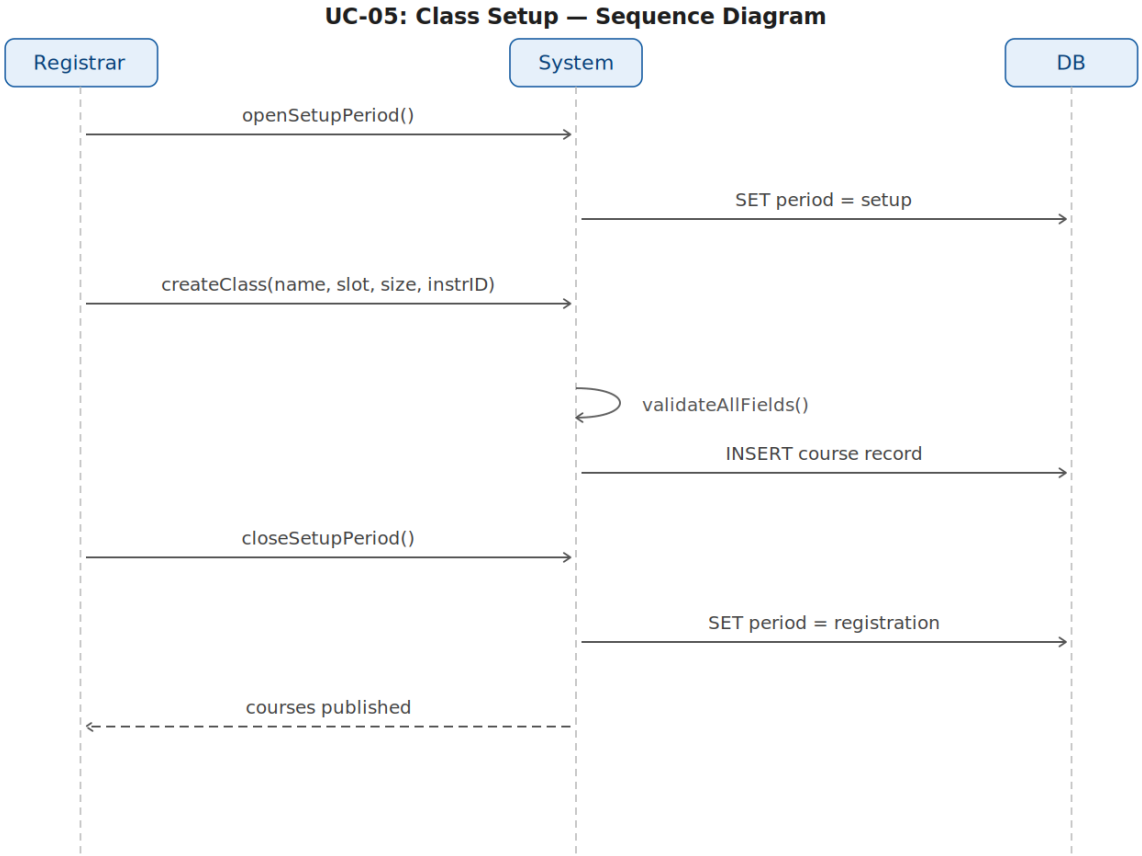
25. A new student logs in using their temporary password for the first time.

26. The system detects first_login = true.
27. The system forces the student to set a new password before any other action. This step cannot be skipped.
28. After the password is changed, the system presents a mandatory onboarding tutorial.
29. After completing the tutorial, the student is directed to their dashboard.

Invalid Credentials

30. The user enters an incorrect username or password.
31. The system fails authentication and displays an error message.
32. The user is not redirected and remains on the login page.

UC-05: Class Setup



Normal Scenario

The Registrar configures all courses for a new semester.

25. The Registrar opens the Class Setup period from the administrative panel.
26. The system enters the Class Setup state.

27. The Registrar creates a new class by specifying the class name, meeting time slot, maximum enrollment size, and assigned instructor.
28. The system validates that all required fields are complete and saves the class.
29. The Registrar repeats this process for all classes in the semester.
30. Once all classes are configured, the Registrar closes the setup period.
31. The system transitions to the Course Registration period and makes all classes visible for student browsing.

Exceptional Scenarios

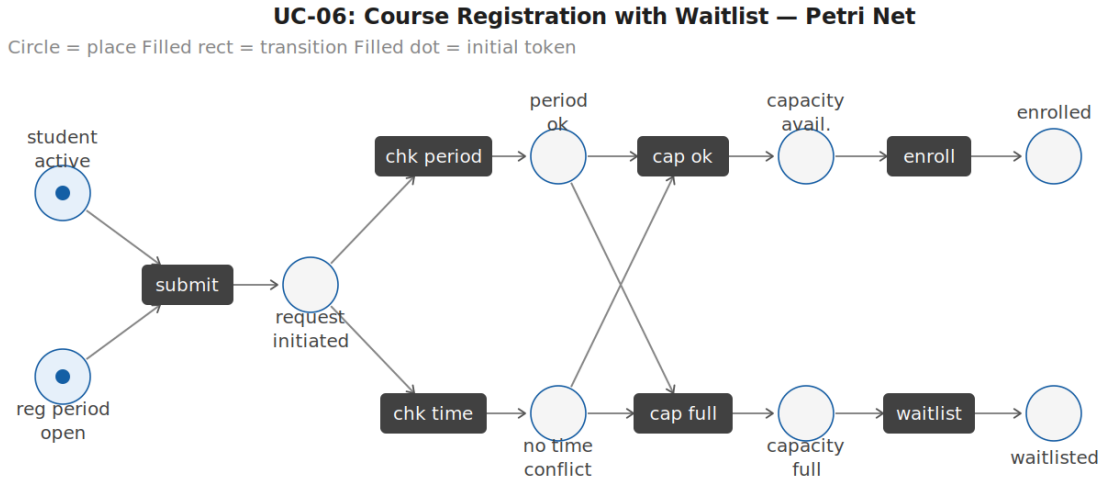
Missing Required Field on Class Creation

33. The Registrar attempts to save a class without specifying an instructor, time slot, or maximum size.
34. The system rejects the save and displays a validation error indicating which field is missing.
35. The Registrar must complete all required fields before the class can be published.

Attempting Class Setup Outside Setup Period

36. A user or process attempts to create or modify class configurations after the setup period has been closed.
37. The system detects the incorrect period state via SystemState.
38. The system blocks the action and returns an error.

UC-06: Register for Courses



Normal Scenario

A student successfully registers for 2–4 courses with no conflicts or capacity issues.

32. The system is in the Course Registration period.
33. The student navigates to the course registration screen.

34. The student browses the catalog and selects 2 to 4 courses.
35. The system verifies that none of the selected courses have overlapping time slots.
36. The system verifies that none of the selected courses are at maximum enrollment.
37. Both checks pass. The system confirms the registration and formally enrolls the student in the selected courses.

Exceptional Scenarios

<u>Selected Course is at Full Capacity</u>

39. The student selects a course that has reached its maximum enrollment.
40. The system informs the student the course is full.
41. The student is placed on that course's waitlist rather than being directly enrolled.
42. The student may continue to register for other available courses.

<u>Time Conflict Between Selected Courses</u>

43. The student selects two or more courses whose scheduled time slots overlap.
44. The system detects the conflict and displays an error.
45. The system does not allow enrollment while the conflict exists.
46. The student must deselect one of the conflicting courses.

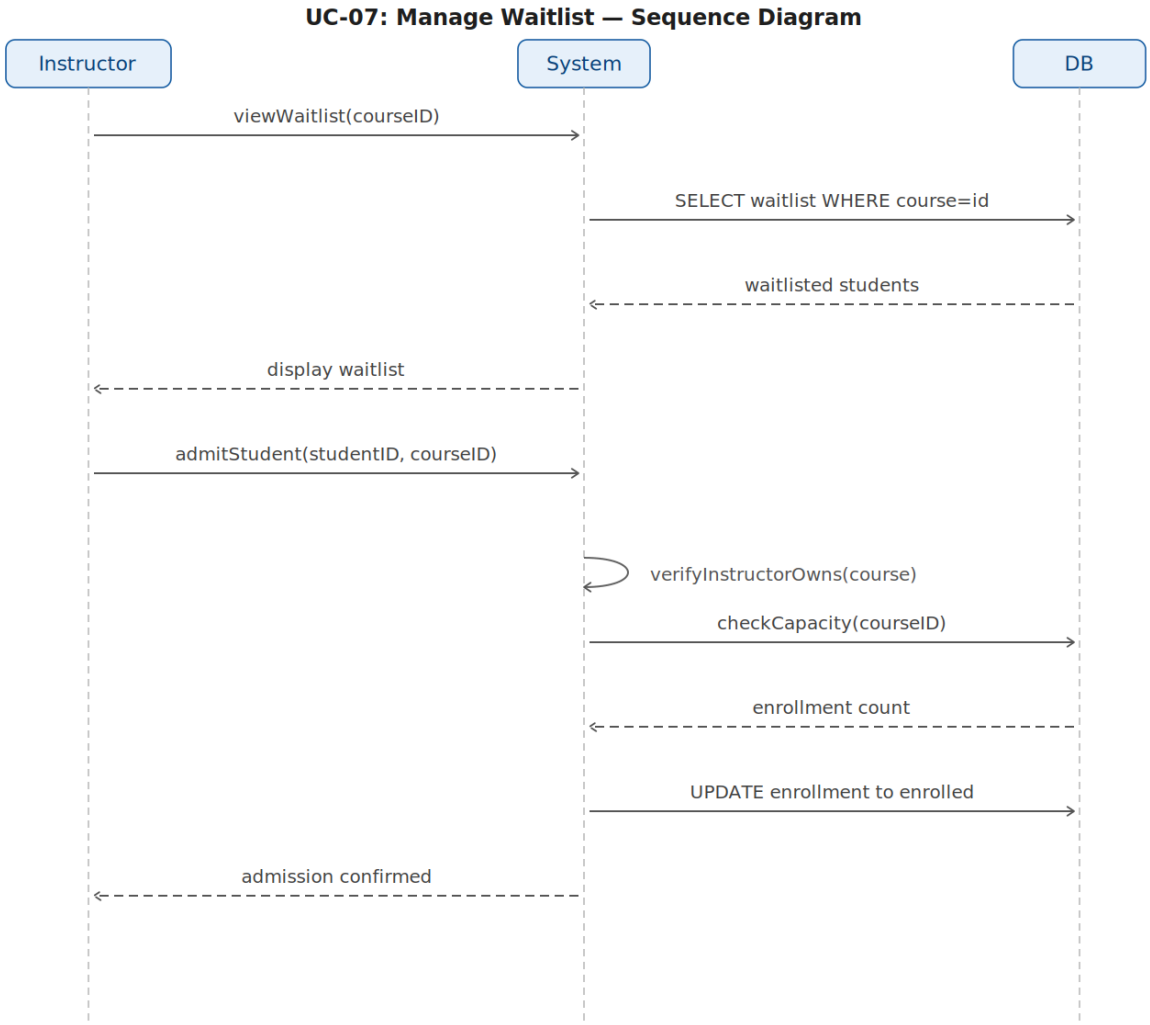
<u>Student Retaking a Previously Failed Course</u>

47. The student selects a course they previously failed with an F grade.
48. The system detects the prior F on the student's record.
49. The system permits re-registration for this specific course as an allowed retake.

<u>Student Attempts to Register for More Than 4 Courses</u>

50. The student tries to add a fifth course to their registration.
51. The system detects the course count would exceed 4.
52. The system returns an error and prevents the addition.

UC-07: Manage Waitlist



Normal Scenario

<u>An instructor admits a waitlisted student to their course.</u>

38. The system is in the Course Registration period.
39. At least one student is on the waitlist for the instructor's course.
40. The instructor navigates to the waitlist management view for their course.
41. The instructor reviews waitlisted students and selects one or more to admit.
42. The system checks that the class is not at maximum capacity.
43. The system moves the selected students from waitlisted to enrolled status.

Exceptional Scenarios

<u>Course Already at Maximum Capacity</u>

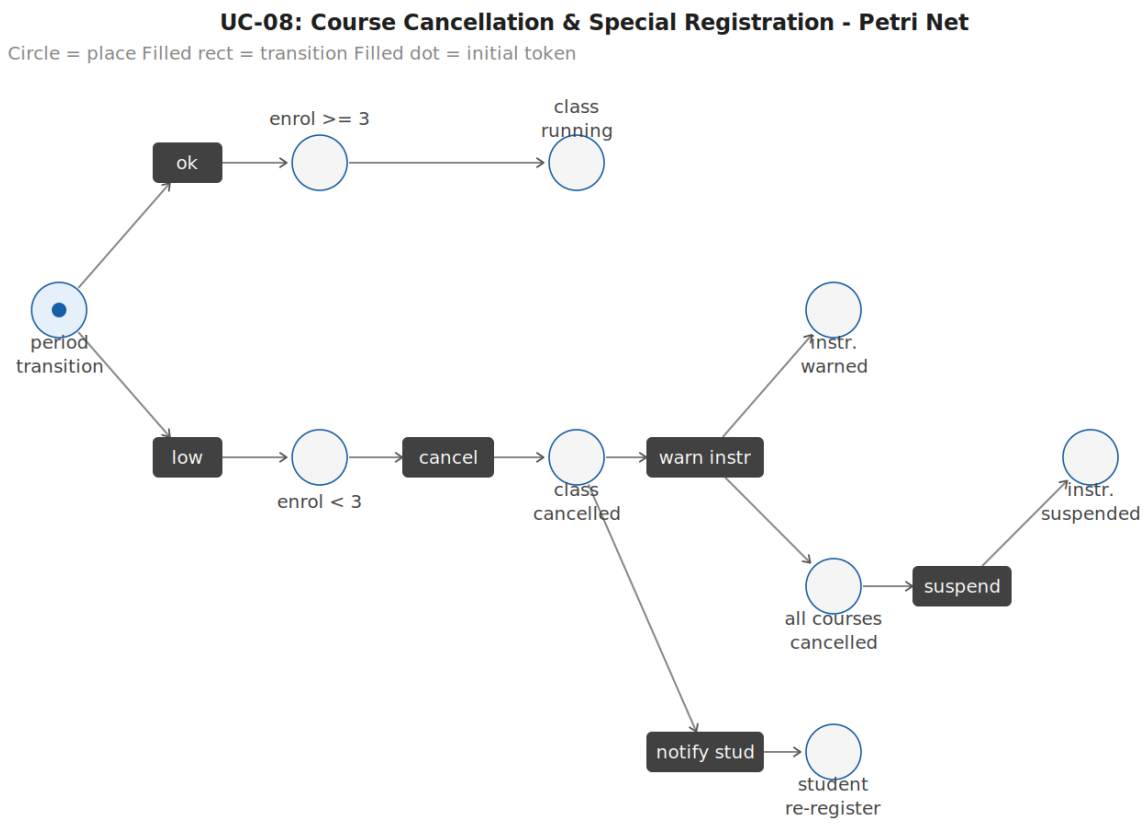
53. The instructor selects a student to admit from the waitlist.

54. The system checks the current enrollment count and finds it already equals MaxSize.
55. The system blocks the admission and notifies the instructor that the class is full.
56. The waitlist remains unchanged.

Another Actor Tries to Admit from Waitlist

57. The Registrar or another user attempts to admit a student from the waitlist on behalf of an instructor.
58. The system rejects the action, as only the assigned course instructor has authority to admit from the waitlist.
59. No enrollment change is made.

UC-08: Course Cancellation and Special Registration



Normal Scenario

The system transitions to Class Running period and automatically cancels under-enrolled courses, allowing affected students to re-register.

44. The Course Registration period closes and the system begins transitioning to the Class Running period.
45. The system automatically checks the enrollment count of every course.

46. All courses have 3 or more enrolled students — no cancellations occur.
47. The system verifies every enrolled student is in at least 2 courses. No warnings are needed.
48. The Class Running period begins normally.

Exceptional Scenarios

Course Cancelled Due to Low Enrollment

60. A course is found to have fewer than 3 enrolled students.
61. The system cancels the course and notifies the course's instructor with a warning.
62. Enrolled students are notified and granted a Special Registration window to select replacement courses from active offerings.
63. Standard registration rules (no time conflicts, capacity checks, 2–4 course limits) apply during Special Registration.
64. If a student falls below 2 enrolled courses after cancellation, the system issues them a warning.

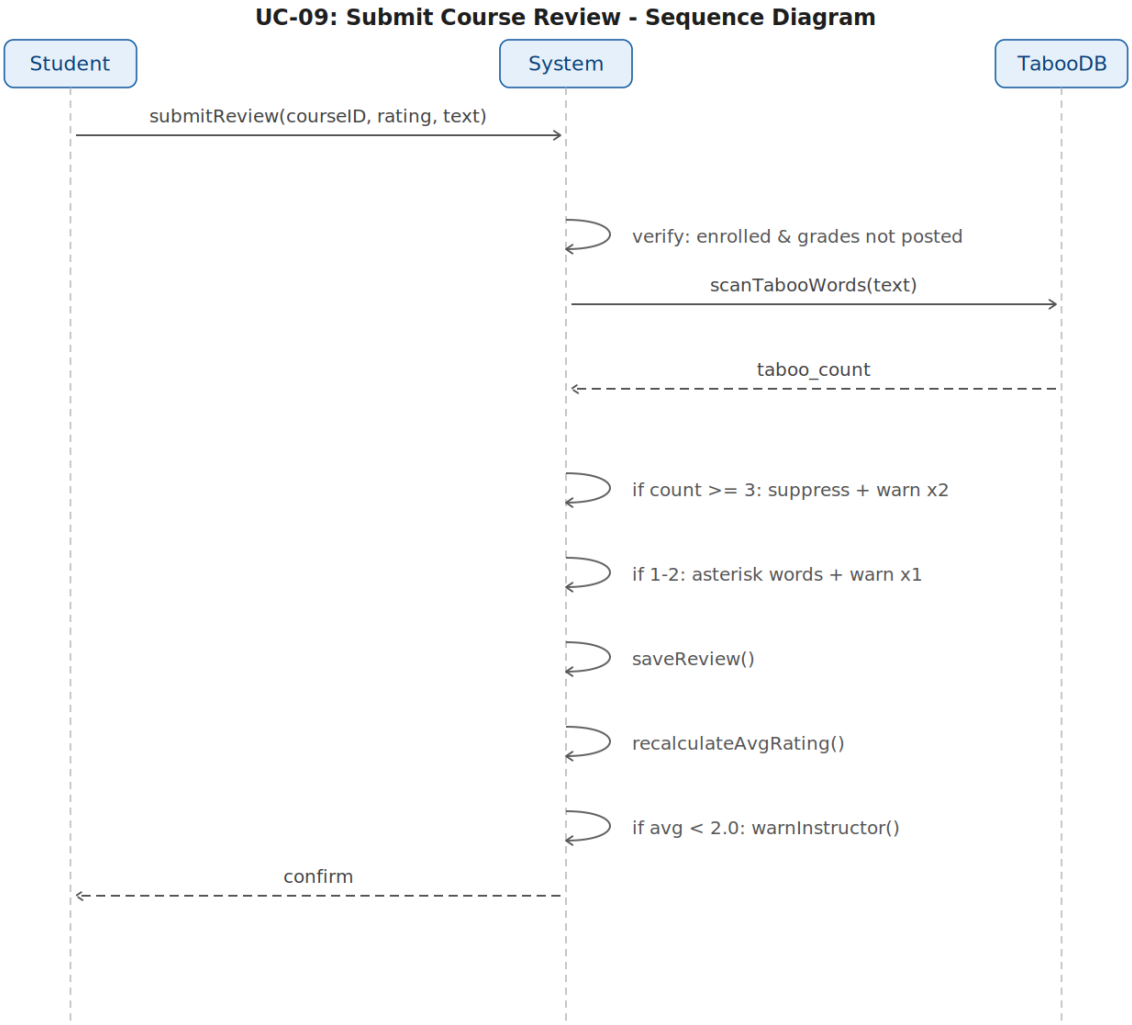
All of an Instructor's Courses Are Cancelled

65. All courses assigned to an instructor are cancelled due to low enrollment.
66. The system issues a warning for each cancelled course.
67. The system flags the instructor as suspended, preventing them from being assigned to teach in the following semester.

Student Falls Below 2 Courses After Cancellation

68. After course cancellations, a student is enrolled in fewer than 2 courses.
69. The system issues the student a warning.
70. The student is permitted to register for replacement courses during the Special Registration window.

UC-09: Submit Course Review and Rating



Normal Scenario

<u>A student submits a clean review with no taboo words.</u>

49. The Class Running period is active and the instructor has not yet posted final grades.
50. The enrolled student navigates to the review submission screen for their course.
51. The student writes a review text and selects a star rating between 1 and 5.
52. The student submits the review.
53. The system scans the review against the active taboo word list. No taboo words are found.
54. The review is published as written.
55. The system recalculates and updates the course's average star rating.

Exceptional Scenarios

<u>Review Contains 1–2 Taboo Words</u>

71. The student submits a review containing one or two taboo words.
72. The system replaces those specific words with asterisks.
73. The filtered review is published.
74. The student receives one warning added to their record.
75. The course average rating is updated.

Review Contains 3 or More Taboo Words

76. The student submits a review containing three or more taboo words.
77. The system suppresses the review entirely — it is not published or shown to anyone.
78. The student receives two warnings added to their record.
79. The course average rating is not updated.

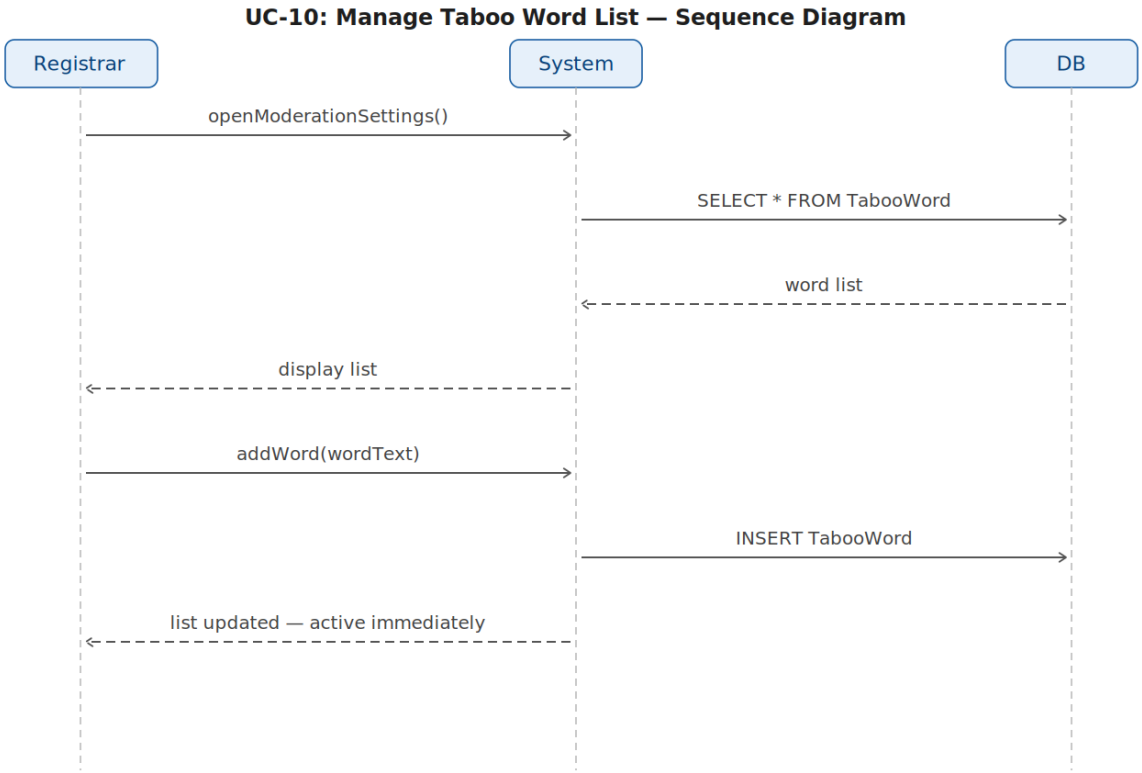
Review Submitted After Grades Are Posted

80. The instructor has already posted final grades for the course.
81. The student attempts to submit a review.
82. The system rejects the submission and returns an error indicating the review window has closed.

Course Average Rating Drops Below 2.0

83. A review is submitted and the system recalculates the course's average star rating.
84. The new average falls below 2.0.
85. The system automatically issues a warning to the course's instructor.

UC-10: Manage Taboo Word List



Normal Scenario

<u>The Registrar adds a new word to the taboo list.</u>

56. The Registrar navigates to the content moderation settings in the system configuration panel.
57. The system displays the current list of taboo words.
58. The Registrar types a new word and submits the addition.
59. The system saves the updated list.
60. The new word is immediately active and will be applied to all reviews submitted from this point forward.

Exceptional Scenarios

<u>Registrar Edits or Removes an Existing Word</u>

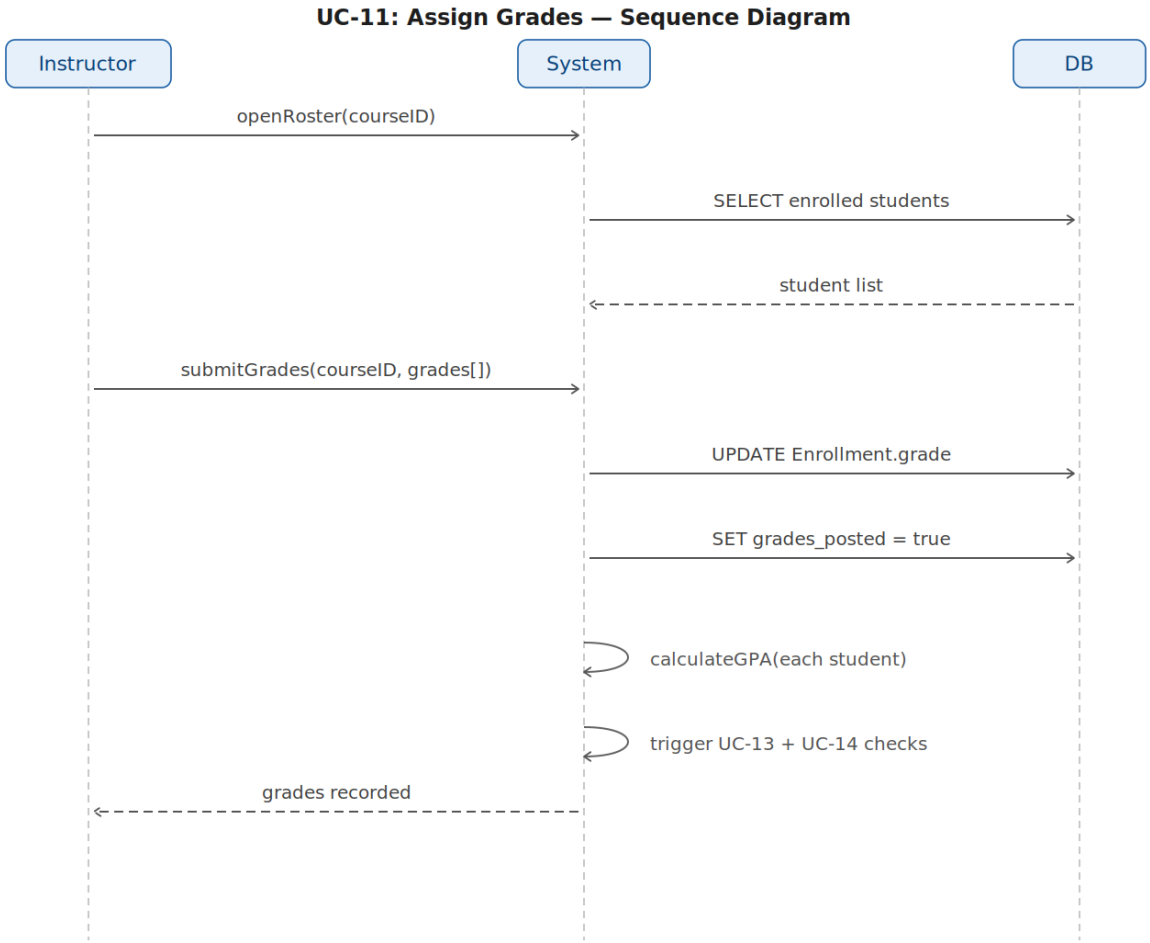
86. The Registrar selects an existing word in the taboo list and modifies or deletes it.
87. The system saves the updated list immediately.
88. Reviews submitted before the change are unaffected — they were already evaluated using the version of the list active at submission time.

<u>Unauthorized User Attempts to Access Taboo Settings</u>

89. A student or instructor navigates to the content moderation settings.
90. The system denies access, as only the Registrar has authority to manage the taboo word list.

91. An appropriate access-denied message is displayed.

UC-11: Assign Grades



Normal Scenario

<u>An instructor assigns final grades to all students before the grading period closes.</u>

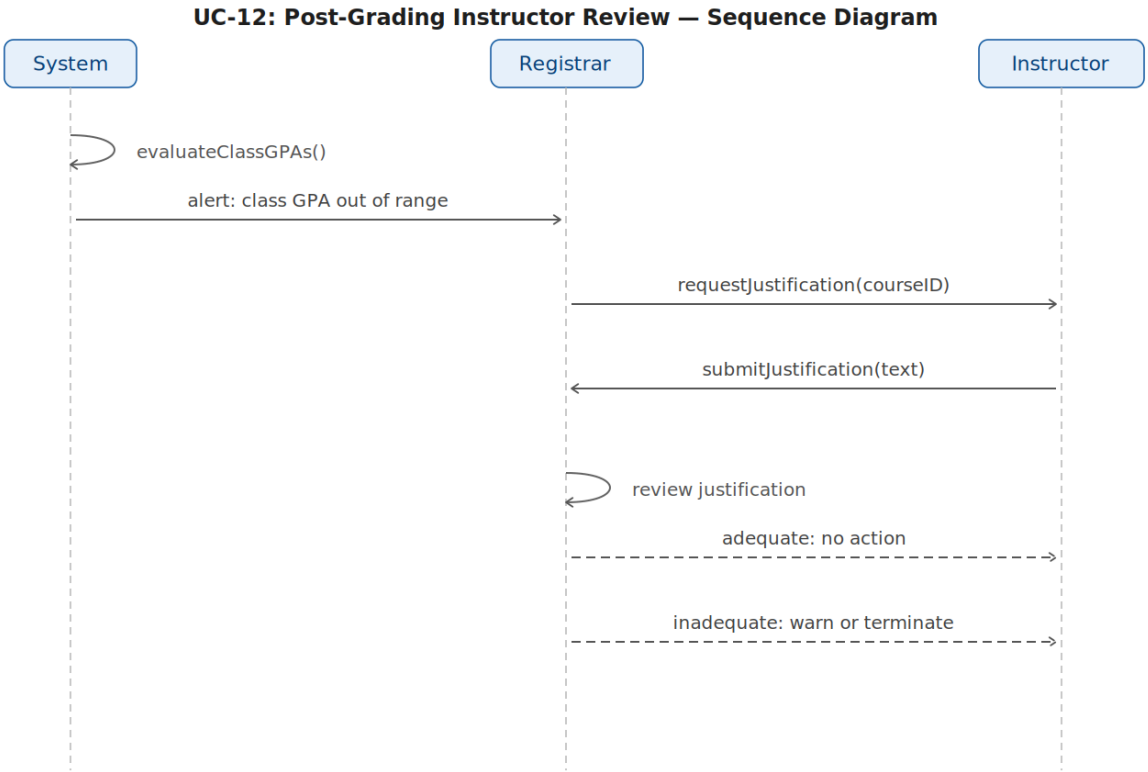
61. The system enters the Grading period.
62. The instructor navigates to their course roster.
63. The instructor assigns a final grade to each enrolled student.
64. The instructor submits all grades.
65. The system records the grades and marks grades_posted = true for the course.
66. The system recalculates semester GPA and cumulative GPA for each affected student.
67. The system automatically triggers the post-grading checks defined in UC-13 and UC-14.

Exceptional Scenarios

<u>Grading Period Closes Before All Grades Are Submitted</u>

92. The grading period deadline passes and the instructor has not assigned grades to all enrolled students.
93. The system automatically issues the instructor one warning for incomplete grading.
94. The system records the warning on the instructor's record.

UC-12: Post-Grading Instructor Review



Normal Scenario

<u>All courses finish with class GPAs in the acceptable range of 2.5–3.5 and no review is triggered.</u>

68. The grading period ends and the system computes final class GPAs for all courses.
69. All class GPAs fall between 2.5 and 3.5 (inclusive).
70. No courses are flagged.
71. No instructor contact is initiated.

Exceptional Scenarios

<u>Class GPA Falls Outside Acceptable Range</u>

95. A course's class GPA is computed as above 3.5 or below 2.5.
96. The system flags the course and sends an alert to the Registrar.
97. The Registrar contacts the instructor and requests a written justification for the grade distribution.

98. The instructor submits a justification.

99. The Registrar reviews the justification and finds it adequate. No further action is taken.

Justification Deemed Inadequate

100. The instructor's justification for the abnormal grade distribution is reviewed by the Registrar and found inadequate.

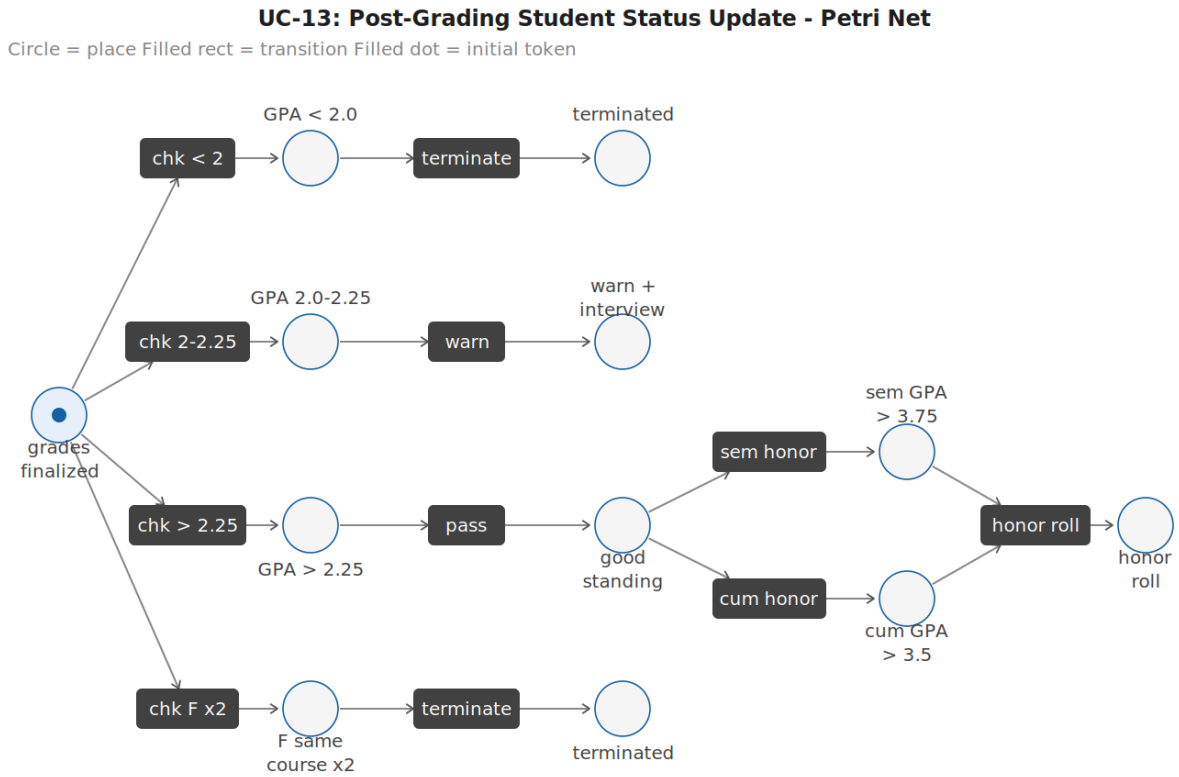
101. The Registrar may issue the instructor a warning, or terminate their employment immediately, at the Registrar's discretion.

Instructor Accumulates 3 Warnings

102. As a result of a warning issued in this or prior reviews, the instructor's total warning count reaches 3.

103. The system automatically suspends the instructor from teaching.

UC-13: Post-Grading Student Status Update



Normal Scenario

A student finishes the semester in good standing with no warnings or special status.

72. All instructors have submitted grades and all GPA calculations are finalized.

73. The system evaluates the student's cumulative GPA and course history.

74. The student's cumulative GPA is between 2.25 and 3.5 with no F grades in the same course twice.

75. No termination or warning criteria are met.

76. The student's status remains active. No changes are applied.

Exceptional Scenarios

Cumulative GPA Below 2.0 — Termination

104. The system evaluates the student's cumulative GPA and finds it below 2.0.

105. The student is automatically terminated and removed from the system.

Student Earns F in the Same Course Twice — Termination

106. The system detects that the student has now received an F in the same course for the second time.

107. The student is automatically terminated and removed from the system.

Cumulative GPA Between 2.0 and 2.25 — Warning and Interview

108. The system evaluates the student's cumulative GPA and finds it between 2.0 and 2.25.

109. The system issues the student one warning.

110. The system flags the student for a mandatory interview with the Registrar.

Student Earns Honor Roll Designation

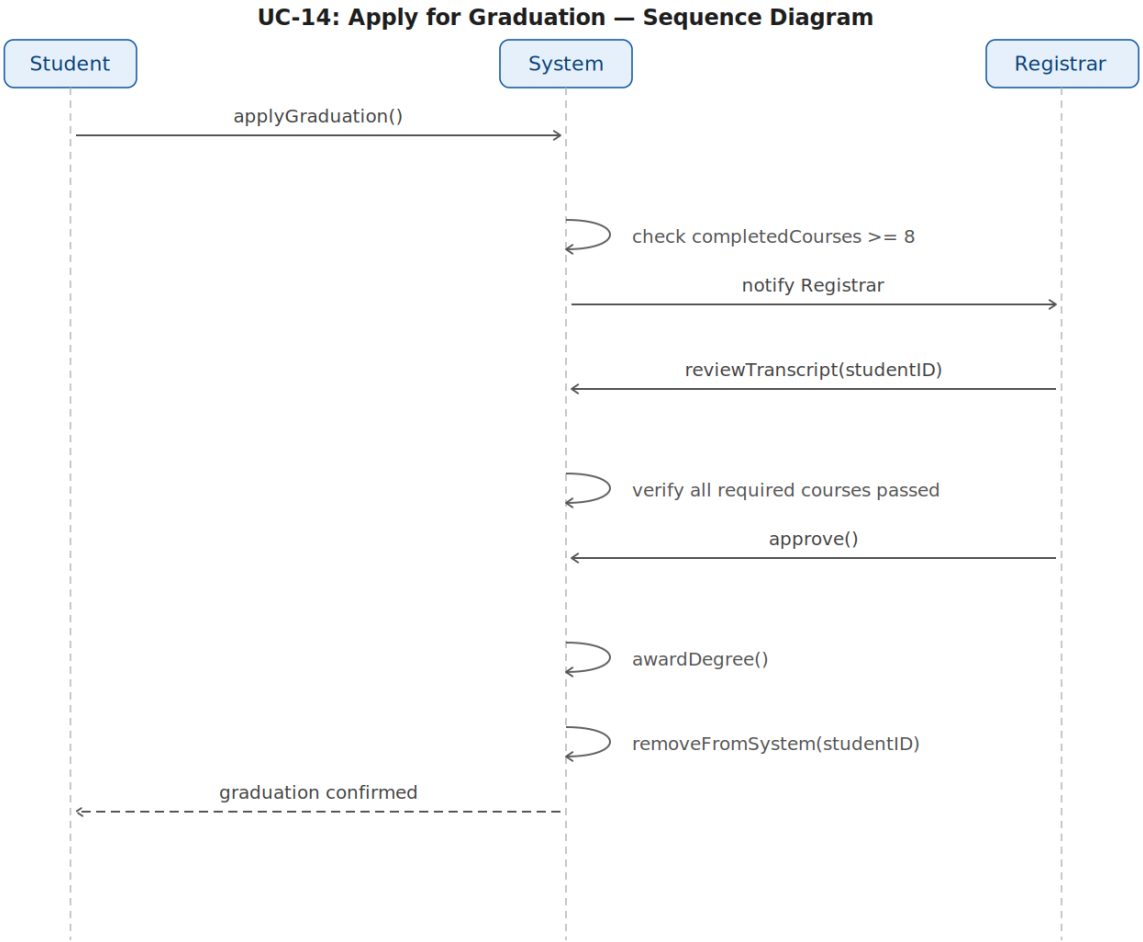
111. The student's semester GPA exceeds 3.75, granting a semester honor roll designation.

112. Or, the student's cumulative GPA exceeds 3.5 after more than one completed semester, granting an overall honor roll designation.

113. The student is labeled with the honor roll status.

114. If the student has at least one existing warning, the system offers them the option to redeem the honor roll designation to remove one warning.

UC-14: Apply for Graduation



Normal Scenario

<u>A student who meets all requirements applies and is graduated.</u>

77. The student has completed at least 8 classes and all program-required courses appear as passed on their transcript.
78. The student navigates to the graduation application section and submits a formal application.
79. The Registrar reviews the student's full transcript.
80. All program requirements are confirmed as satisfied.
81. The Registrar approves the graduation.
82. The student is awarded a Bachelor's degree and their account is removed from the active system.

Exceptional Scenarios

<u>Not All Required Courses Completed</u>

115. The Registrar reviews the transcript and finds one or more program-required courses have not been completed or passed.

116. The graduation application is denied.

117. The student receives one warning for submitting a reckless graduation application.

118. The student remains active in the system.

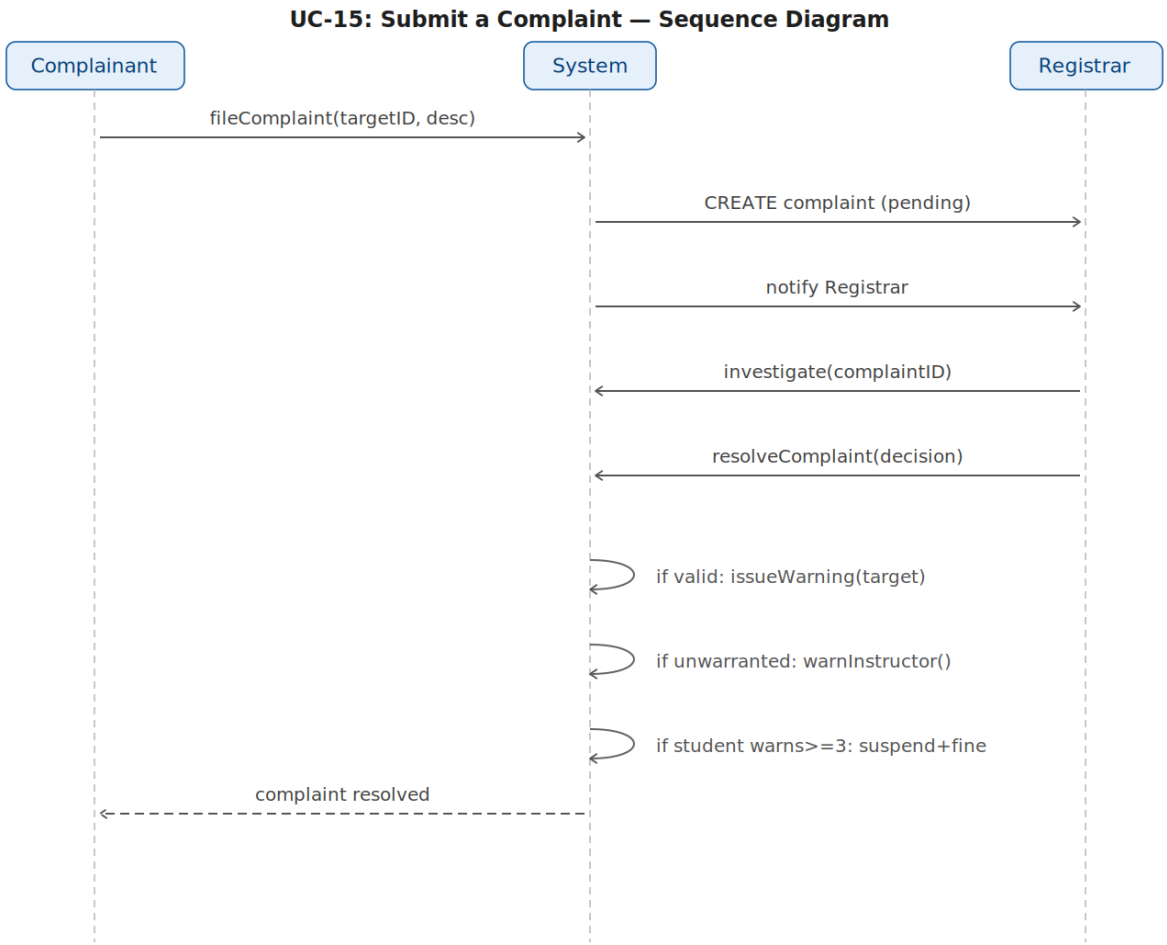
Student Has Fewer Than 8 Completed Classes

119. The student attempts to submit a graduation application despite having completed fewer than 8 classes.

120. The system detects the completion count does not meet the minimum threshold.

121. The system blocks the submission and returns an error. The student cannot submit the application.

UC-15: Submit a Complaint



Normal Scenario

A student files a complaint against another student and the Registrar resolves it.

83. The student navigates to the complaint submission form.
84. The student selects the target user (another student) and writes a description of the issue.
85. The system records the complaint as pending and routes it to the Registrar.
86. The Registrar reviews the complaint and investigates.
87. The Registrar determines the complaint is valid and issues a warning to the accused student.
88. The complaint is marked resolved with a documented outcome.

Exceptional Scenarios

Instructor Files Complaint Against a Student — Found Valid

122. An instructor submits a formal complaint against a student.
123. The Registrar reviews and determines the complaint is valid.
124. The Registrar takes a concrete action: either issuing the student a warning or de-registering them from a course.
125. The complaint is marked resolved.

Instructor Files Complaint Against a Student — Found Unwarranted

126. The Registrar reviews an instructor's complaint against a student and determines it is unwarranted.
127. The Registrar issues one warning to the complaining instructor instead.
128. The student faces no action.

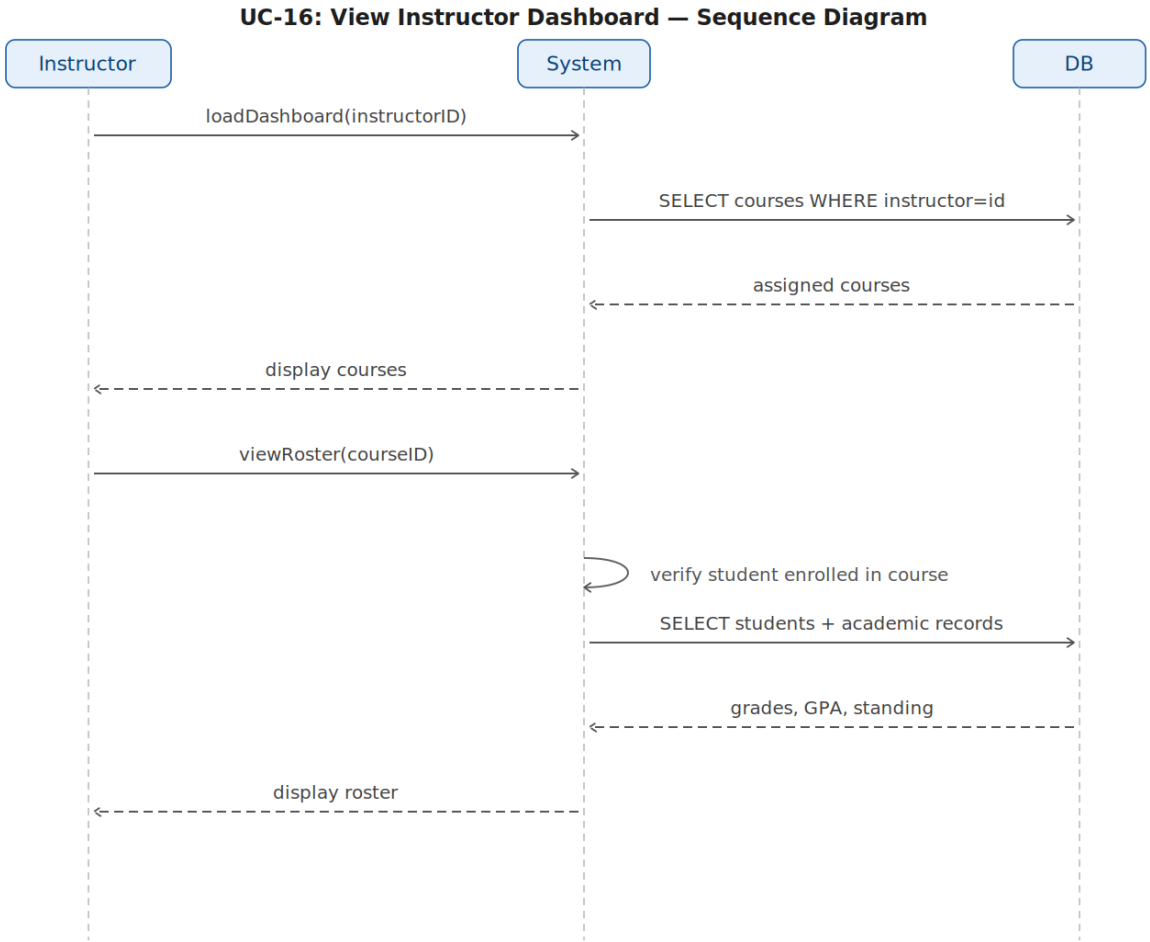
Student Accumulates 3 or More Total Warnings

129. As a result of any combination of warnings (complaints, reviews, academic standing, etc.), a student's total warning count reaches 3 or more.
130. The system automatically suspends the student for one semester.
131. The system requires the student to pay a fine to the Registrar before re-enrollment.

Student Attempts to File Complaint Against an Instructor

132. A student files a complaint against an instructor.
133. The Registrar reviews the complaint.
134. The Registrar may issue a warning to the instructor if the complaint is found valid, or take no action if it is deemed unwarranted.

UC-16: View Instructor Dashboard



Normal Scenario

<u>An instructor views their assigned courses and a student's academic record.</u>

89. The instructor logs in and is routed to their dashboard.
90. The dashboard displays all courses currently assigned to the instructor for the active semester.
91. The instructor selects a course and views the full student roster.
92. The instructor selects a student from the roster.
93. The system displays the student's basic personal information, grades, GPA, and academic standing.

Exceptional Scenarios

<u>Instructor Attempts to View a Student Outside Their Courses</u>

135. The instructor attempts to access the academic record of a student not enrolled in any of their active courses.
136. The system denies access and returns an authorization error.

137. The instructor may only view records within their assigned active courses.

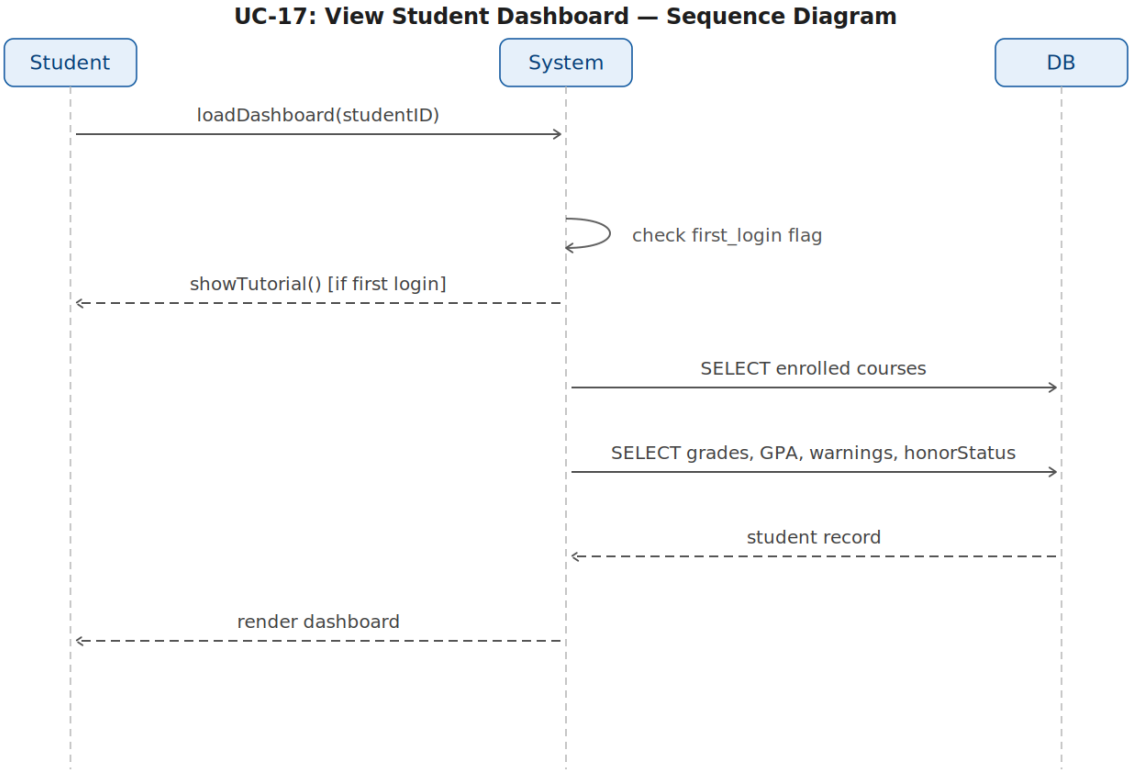
Instructor Has No Assigned Courses for the Semester

138. The instructor logs in but has no courses assigned for the current semester (e.g., suspended or not yet assigned).

139. The dashboard displays an empty course list.

140. No student records are accessible.

UC-17: View Student Dashboard



Normal Scenario

A returning student views their dashboard after completing the first-login flow.

94. The student logs in with their credentials (first_login = false).

95. The system routes the student to their personal dashboard.

96. The dashboard displays currently enrolled courses, all historical grades, current semester GPA, cumulative GPA, any active warnings, and honor roll status if applicable.

97. The student navigates to other features such as course registration or review submission from the dashboard.

Exceptional Scenarios

First Login — Mandatory Tutorial

141. A new student logs in for the first time after changing their temporary password.

142. The system detects first_login = true and presents the mandatory onboarding tutorial.

143. The student must complete the tutorial before the dashboard is displayed. The tutorial cannot be skipped.

144. After completing the tutorial, the student is directed to their dashboard.

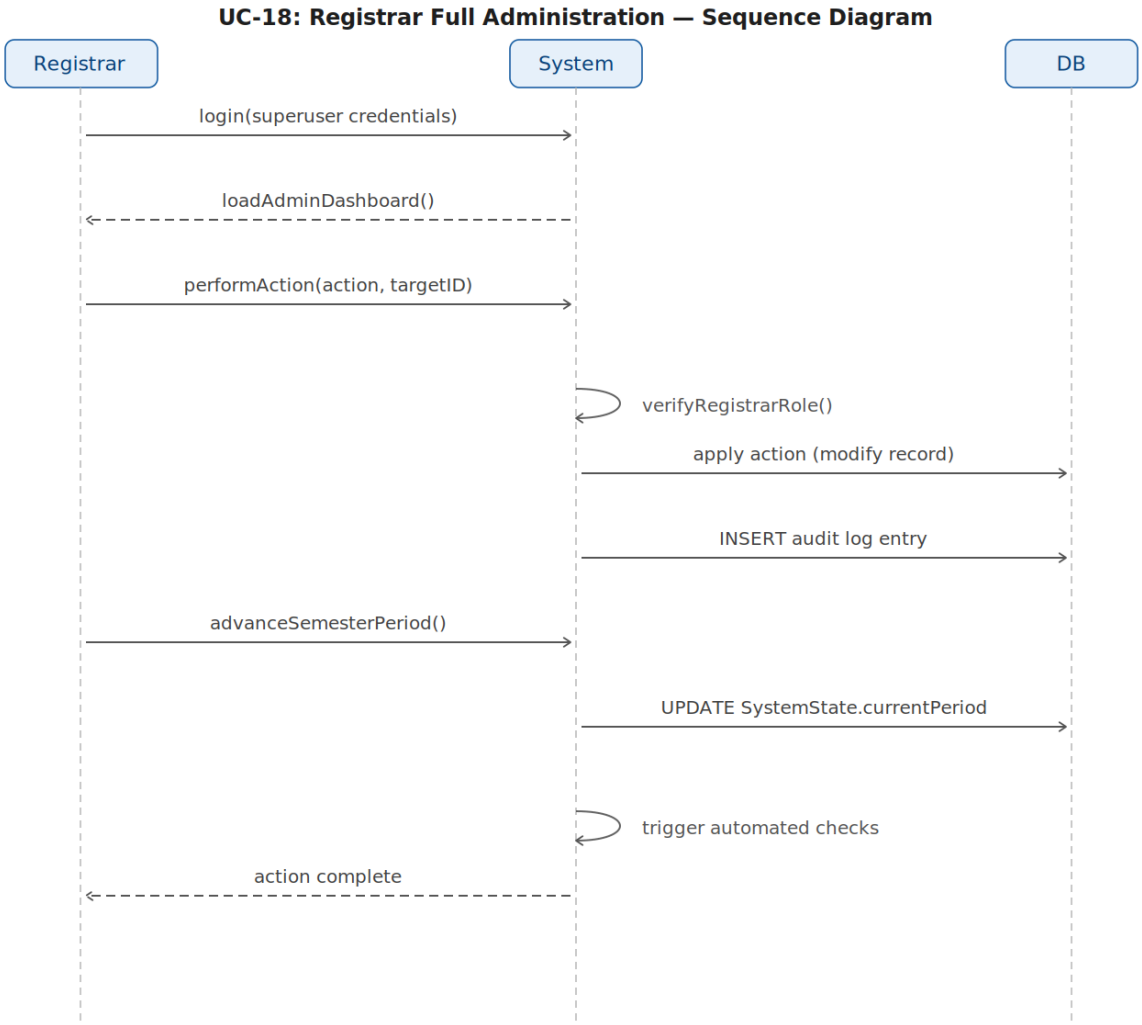
Student Has Active Warnings or Suspension Flag

145. The student's record has one or more active warnings.

146. The dashboard prominently displays the warning count and any associated status flags (e.g., mandatory Registrar interview).

147. If the student's warning count has reached 3 or more, a suspension notice is displayed.

UC-18: Registrar Full Administration



Normal Scenario

<u>The Registrar logs in and performs a standard administrative action.</u>

98. The Registrar logs in with superuser credentials.
99. The system routes the Registrar to the full administration dashboard.
100. The Registrar accesses any student, instructor, or course record as needed.
101. The Registrar performs an administrative action — such as advancing the semester period, approving an application, or resolving a complaint.
102. The system records the action in the audit log for the modified record.

Exceptional Scenarios

<u>Advancing the Semester Period Triggers Automated Processes</u>

148. The Registrar advances the semester from the Registration period to the Class Running period.

149. The system automatically triggers UC-08 logic: checking course enrollment, cancelling under-enrolled courses, issuing instructor warnings, and opening the Special Registration window for affected students.

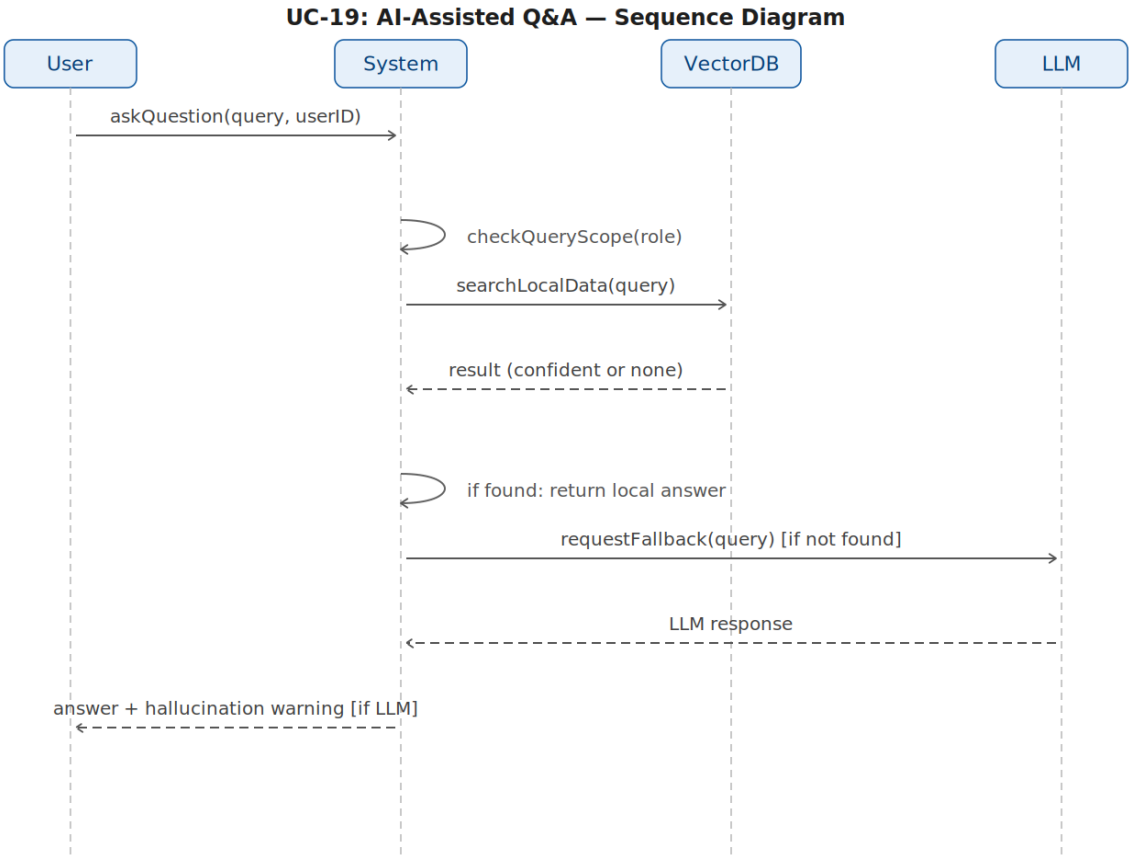
150. The Registrar monitors the results via the administration dashboard.

Registrar Modifies a Record Without an Audit Entry

151. Any modification to a student or instructor record must be logged.

152. If a logging failure occurs, the system should surface an error and prevent the write from completing without an audit trail.

UC-19: AI-Assisted Q&A



Normal Scenario

A student asks a question that is answered from the local vector database.

103. The student navigates to a page with the AI text area.

104. The student types a natural language question about their enrolled courses.

105. The system queries the local vector database for College0-specific information.

106. A sufficiently confident match is found in the local database.

107. The system returns the locally-sourced answer to the student. No hallucination warning is shown.

Exceptional Scenarios

No Local Answer Found — Fallback to External LLM

153. The student submits a question.

154. The system queries the local vector database and does not find a sufficiently confident answer.

155. The system forwards the query to the external LLM.

156. The LLM returns a response.

157. The system displays the answer to the student along with a clearly visible hallucination warning.

Visitor Attempts to Ask a Scoped Question

158. An unauthenticated Visitor submits a question about a specific student's record or enrolled course details.

159. The system enforces scope restrictions — Visitors may only ask general questions about classes and program requirements.

160. The system rejects or redirects the query accordingly.

Local Vector Database or LLM Service is Unavailable

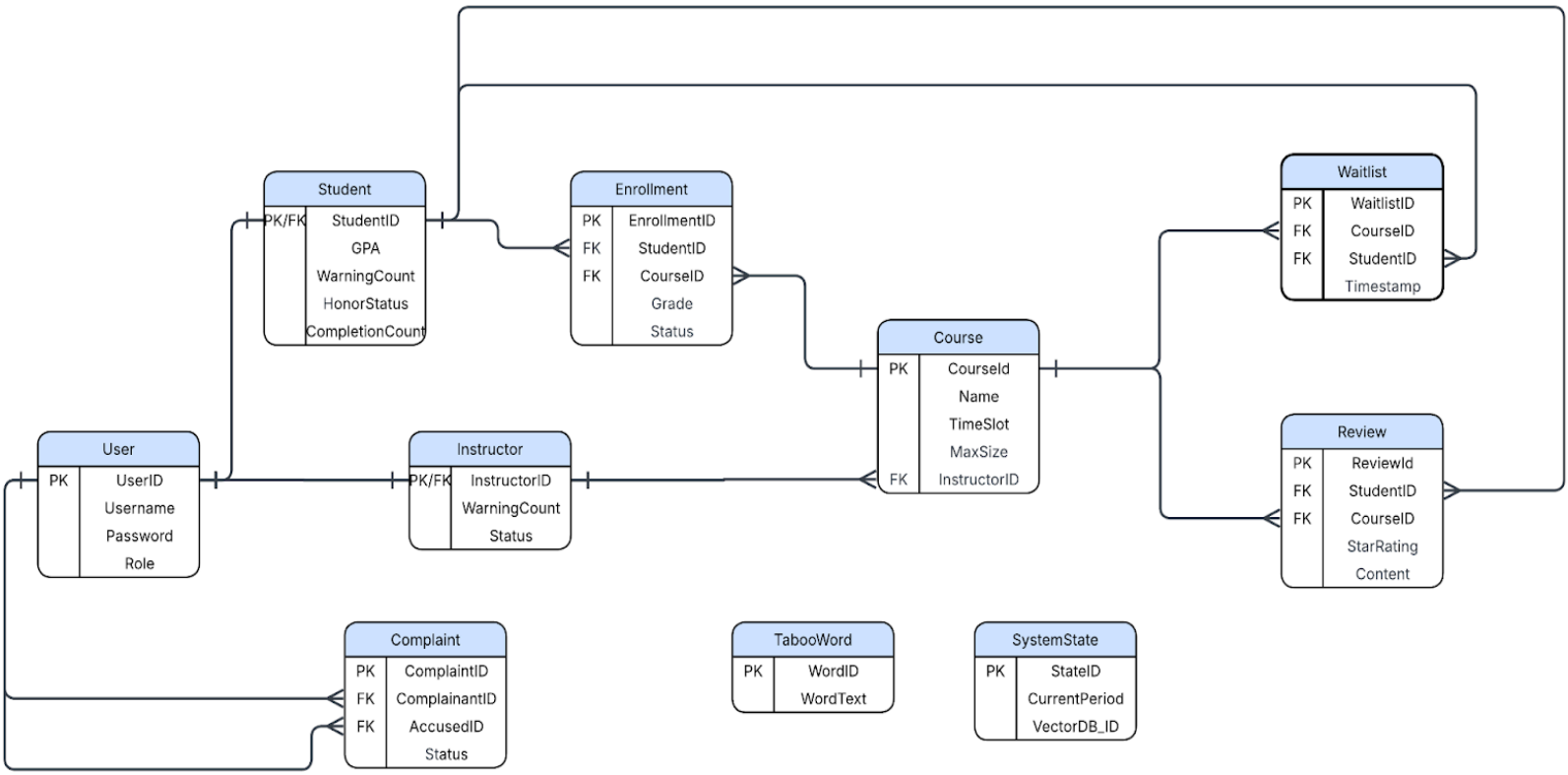
161. The user submits a question.

162. The system attempts to query the local vector database but the service is unavailable.

163. The system may attempt the external LLM fallback.

164. If both services are unavailable, the system displays an error message indicating the AI assistant is currently unavailable.

3. ER Diagram



3.1 Overview

The College0 database is designed as a relational system tailored for a "Toy System" scale, optimized to manage approximately 10 students and their academic lifecycle. The architecture focuses on robust logic for automated warnings, GPA-based status updates, and AI-enabled querying.

3.2 Entity Catalog

The following table defines the core entities, their primary/foreign keys, and their roles within the system logic:

Entity	Attributes	Logic & Purpose

User	**UserID (PK), Username, Password, Role**	The base authentication table for all four user roles: Registrar, Instructor, Student, and Visitor.
Student	**StudentID (PK/FK), GPA, WarningCount, HonorStatus, CompletionCount**	Tracks academic standing, cumulative GPA for termination/honor roll logic, and counts of completed courses for graduation.
Instructor	**InstructorID (PK/FK), WarningCount, Status (Active/Suspended)**	Monitors teaching eligibility and tracks disciplinary warnings.
Course	**CourseID (PK), Name, TimeSlot, MaxSize, InstructorID (FK)**	Defines class parameters and assigns specific faculty to lead each course.
Enrollment	**EnrollmentID (PK), StudentID (FK), CourseID (FK), Grade, Status**	An intermediary table managing active student-course links and historical grades.
Waitlist	**WaitlistID (PK), CourseID (FK), StudentID (FK), Timestamp**	Tracks students awaiting instructor approval when a course reaches its maximum size.
Review	**ReviewID (PK), StudentID (FK), CourseID (FK), StarRating, Content**	Stores student feedback. Content is subject to automated moderation against the TabooWord list.

Complaint	**ComplaintID (PK), ComplainantID (FK), AccusedID (FK), Description, Status**	A self-referencing table allowing any user to file grievances against another for Registrar review.
TabooWord	**WordID (PK), WordText**	A global reference list managed by the Registrar to filter reviews and issue automated warnings.
SystemState	**StateID (PK), CurrentPeriod, VectorDB_ID**	A standalone table governing global system behavior, such as the active semester period and AI grounding data.

3.3 Relationship & Cardinality Logic

To ensure data integrity, the following relationships are established using **Crow's Foot Notation**:

- **Role Identification (1:1):** The User table links to Student or Instructor on a one-to-one basis, ensuring each unique login maps to a single academic profile.
- **Instruction Management (1:N):** One Instructor can be assigned to many Course records, but each course is managed by exactly one instructor.
- **Academic Enrollment (1:N):** Student and Course tables both link to the Enrollment table. This tracks the requirement that students must enroll in 2–4 courses per semester.
- **Feedback & Support (1:N):** Students can generate multiple Review and Waitlist entries, each linked back to a specific CourseID to maintain context for instructor approval or rating summaries.
- **Disciplinary Interface (1:N):** The Complaint table uses two foreign keys (ComplainantID and AccusedID) pointing back to the User table, enabling any user to be involved in multiple grievances.

3.4 Support for Business Rules

- **Automated Warnings:** The WarningCount attributes in Student and Instructor allow the logic to trigger suspensions at 3 warnings.
- **Graduation Verification:** The CompletionCount in the Student entity directly supports the UC-14 requirement for 8 completed classes prior to graduation.

- **AI Grounding:** The SystemState entity stores the VectorDB_ID, ensuring the AI Assistant prioritizes institutional data before falling back to general LLM services.
- **Period Control:** The CurrentPeriod in SystemState acts as a global guardrail, blocking actions like course registration during the "Running" or "Grading" periods.

4. Detailed Design (Psuedo-Code)

Authentication and User Management

1) login(user_id, password)

```
        IF user exists AND password matches THEN
                IF first_login = true THEN
                        changePassword()
                ENDIF
                RETURN success
        ELSE
                RETURN error
        ENDIF
```

2) changePassword(user_id, new_password)

```
        UPDATE user.password = new_password
        SET first_login = false
        RETURN success
```

Application

1) apply(type, applicant_data)

```
        CREATE Application(status = “pending”)
        STORE applicant_data
        RETURN confirmation
```

2) reviewApplication(application_id, decision, registrar_id, justification=NULL)

```
        GET application

        IF decision = “accept” THEN
                IF application.type = “student” THEN
                        IF GPA > 3.0 AND quota NOT reached THEN
                                ACCEPT
                        ELSE
                                REQUIRE justification
```

```
                LOG override
            ENDIF
            CREATE student account
        ELSE IF application.type = "instructor" THEN
            CREATE instructor account
            ASSIGN classes
        ENDIF
        GENERATE user_id, temp_password
        SET first_login = true
    ELSE
        SET status = "rejected"
    ENDIF

    SAVE decision
```

Semester Management

1) setupSemester(data)

```
    CREATE semester
    SET period = "setup"
```

2) advanceSemesterPeriod()

```
    IF period = "setup" THEN
        period = "registration"
    ELSE IF period = "registration" THEN
        period = "running"
    ELSE IF period = "running" THEN
        period = "grading"
    ENDIF
```

Course and Class Management

1) createCourse(course_data)

```
    INSERT course
```

2) createClass(course_id, instructor_id, capacity, schedule)

```
    CREATE class
    ASSIGN instructor
    SET capacity and schedule
    SET status = "open"
```

Course Registration

```
1) registerCourse(student_id, class_id)
        IF current_period != "registration" THEN
            RETURN error
        IF current_course_count >= 4 THEN
            RETURN error
        IF timeConflict(student_id, class_id) THEN
            RETURN error
        IF class.capacity NOT full THEN
            CREATE Enrollment(status = "enrolled")
        ELSE
            CREATE Enrollment(status = "waitlisted")
        ENDIF

2) timeConflict(student_id, class_id)
        FOR each enrolled class
            IF schedules overlap THEN
                RETURN true
        ENDFOR
        RETURN false

3) manageWaitlist(instructor_id, student_id, class_id)
        IF instructor owns class THEN
            MOVE student from waitlist to enrolled
```

Class Running Period

```
1) checkCourseStatus()
        FOR each class
            IF enrolled_students < 3 THEN
                CANCEL class
                NOTIFY students
                ALLOW re-registration window
            ENDIF
        ENDFOR

2) checkInstructorStatus(instructor_id)
        IF instructor has 0 active classes THEN
            SUSPEND instructor for next semester

3) checkStudentLoad(student_id)
        IF enrolled_courses < 2 THEN
```

```
                ISSUE warning
```

Review System

```
1)  submitReview(student_id, class_id, rating, comment)
        IF current_period != "running OR grades_posted THEN
            RETURN error

        taboo_count = countTabooWords(comment)

        IF taboo_count >= 3 THEN
            REJECT review
            issueWarning(student_id, "taboo language", 2)
        ELSE
            CLEAN comment (* replace taboo words)
            IF taboo_count >= 1 THEN
                issueWarning(student_id, "taboo language", 1)
            ENDIF
            SAVE review
        ENDIF

2)  calculateRating(class_id)
        RETURN average(review.rating)

3)  checkInstructorRating(instructor_id)
        FOR each class taught
            IF average_rating < 2 THEN
                issueWarning(instructor_id, "low rating", 1)
            ENDIF
        ENDFOR

        IF warning_count >= 3 THEN
            SUSPEND instructor
```

Grading System

```
1)  submitGrades(instructor_id, class_id, grades)
        VERIFY instructor teaches class
        FOR each student
            UPDATE Enrollment.grade
        ENDFOR
        SET grades_posted = true
```

2) checkInstructorGrading(instructor_id)

```
        IF an class missing grades THEN
            issueWarning(instructor_id, “missing grades”, 1)
        ENDIF
```

3) calculateGPA(student_id)

```
        SUM grade_points / total_courses
        RETURN GPA
```

4) evaluateClassGPA(class_id)

```
        IF class_GPA < 2.5 OR class_GPA > 3.5 THEN
            FLAG for registrar review
```

Student Academic Standing

1) evaluateStudentStatus(student_id)

```
        GPA = calculateGPA(student_id)

        IF GPA < 2 OR failed_same_course_twice THEN
            TERMINATE student
        ELSE IF GPA > 2 but < 2.25 THEN
            issueWarning(student_id, “low GPA”, 1)
            REQUIRE registrar interview
        ENDIF
```

2) applyHonorRoll(student_id)

```
        IF semester_GPA > 3.75 OR cumulative_GPA > 3.5 THEN
            MARK honor_roll

            IF warning_count > 0 THEN
                REMOVE 1 warning
            ENDIF
        ENDIF
```

Graduation

1) applyGraduation(student_id)

```
        IF completed_courses >= 8 AND required_courses_met THEN
            GRADUATE student
            REMOVE from system
        ELSE
            issueWarning(student_id, “invalid graduation request”, 1)
        ENDIF
```

Warning System

1) issueWarning(user_id, reason, count=1)

```
INCREMENT warning_count by count

IF warning_count >= 3 THEN
    SUSPEND user
    APPLY fine
ENDIF
```

Complaint System

1) fileComplaint(user_id, target_id, description)

```
CREATE Complaint(status = "pending")
```

2) resolveComplaint(complaint_id, decision)

```
IF decision = "valid" THEN
    issueWarning(target_user)
ELSE
    issueWarning(complainant)
ENDIF
```

3) handleInstructorComplaint(instructor_id, student_id)

```
IF complaint is valid THEN
    DEREGISTER or WARN student
ELSE
    issueWarning(instructor_id)
ENDIF
```

Dashboard System

1) viewDashboard(user_id)

```
IF role = student THEN
    SHOW GPA, courses, warnings
    IF first_login THEN SHOW tutorial
ELSE IF role = instructor THEN
    SHOW classes, student records
ELSE IF role = registrar THEN
    SHOW full system data
ENDIF
```

AI System

1) askAI(user_id, query)

```
SEARCH vector_database

IF found THEN
    RETURN local answer
ELSE
    CALL LLM
    RETURN answer
ENDIF
```

5. System Screens (GUI Design & Prototype)

This section presents the major graphical user interface (GUI) screens of the College0 system. Each screen includes its purpose, key components, and interactions. A sample prototype flow is also described.

5.1 Login Page

Purpose: Authenticate users and grant access based on roles

Components:

- Username field

- Password field

- Login button

- Error message display

User Actions:

- Enter credentials

- Click login

- Redirect to dashboard

5.2 Student Dashboard

Purpose: Provide overview of student data and actions

Components:

- Enrolled courses

- GPA display

- Registration button

- AI Chat button

User Actions:

- View courses

- Register for courses

- Ask AI

5.3 Instructor Dashboard

Purpose: Allow instructors to manage courses and grades

Components:

- Course list

- Student list

- Grade entry form

User Actions:

- Select course

- Assign grades

- Submit grades

5.4 Registrar Dashboard

Purpose: Provide full system control and administrative actions

Components:

- Applications list

- Complaints panel

- Semester controls

User Actions:

- Approve/reject applications

- Resolve complaints

- Manage semesters

5.5 Course Registration Screen

Purpose: Allow students to select courses

Components:

- Course catalog

- Schedule view

- Register button

User Actions:

- Select courses

- Check conflicts

- Submit registration

5.6 Review Submission Screen

Purpose: Allow students to submit course reviews

Components:

- Text input box

- Submit button

User Actions:

- Write review

- Submit

- System filters taboo words

5.7 AI Chat Interface

Purpose: Allow users to ask questions

Components:

- Text input

- Submit button

- Response area

User Actions:

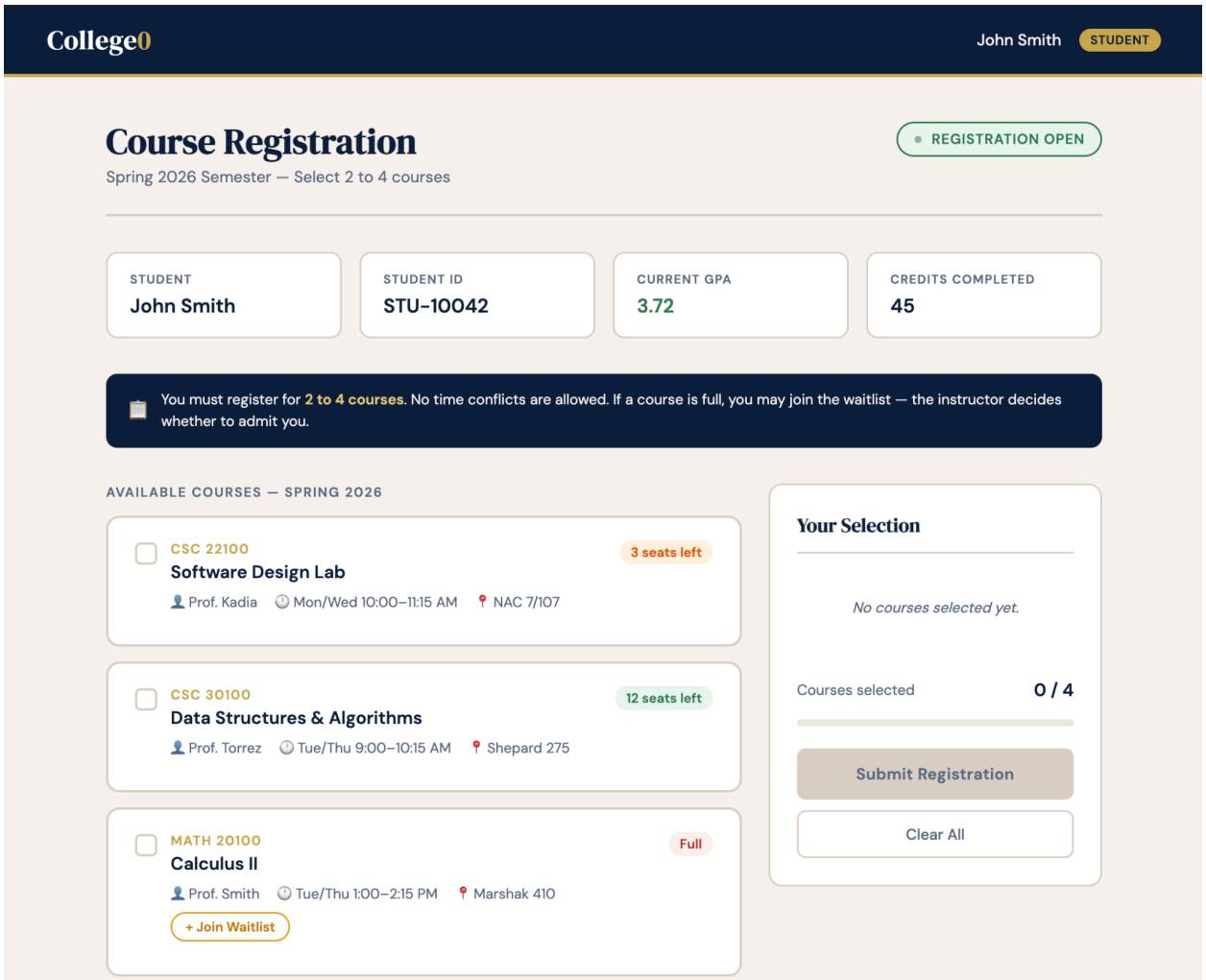
- Type question

- Receive answer

- View warning if external AI used

5.8 Sample Prototype: Course Registration Functionality

The sample prototype selected for this system is the Course Registration functionality. It demonstrates how a student interacts with the system to select and register for courses.



Functionality Steps:
1. Student logs in
2. Navigates to registration screen

3. Selects courses
4. System checks constraints
5. Displays result

6. Memos of Group Meetings and Possible Concerns of Team Work

The following memos document the group meetings held throughout the development of College0. All five members of the team were present at each meeting.

Meeting Memo #1 — Project Kickoff and Requirements Analysis

Date: 3/5/2026 at 3:30 PM

Location: On Campus Library

Attendees: Tristan Chong, Eddie Zhu, Decong Deng, Sangita Shrestha, Gaurab Pun

Discussion Summary:

- The team read through the full College0 project specification together. Each requirement was discussed to ensure all members had a shared understanding of the system scope.
- The four user roles were identified and discussed in detail: Registrar (super user), Instructor, Student, and Visitor. The team clarified the access permissions and responsibilities of each role, particularly the elevated authority of the Registrar over all system operations.
- The four semester periods - class set-up, course registration, class running, and grading were mapped out as a lifecycle. The team noted that many business rules (e.g., course cancellation, waitlisting, student warnings) are period-dependent and must be enforced with state tracking.
- The student application and enrollment workflow was discussed: auto-acceptance rules based on GPA > 3.0 and program quota, the Registrar's override capability with justification, and the unique student ID and temporary password assignment for new students.
- The team identified the AI feature as a significant technical challenge requiring a vector database and LLM integration, and agreed to research available tools (e.g., ChromaDB, FAISS, OpenAI API) before the next meeting.
- Initial thoughts on tech stack were discussed. Front-end options (React, plain HTML/JS) and back-end options (Python/Flask, Node.js) were proposed. No final decision was made.

Concerns / Action Items:

- The scope of the project is large. The team is concerned about managing time effectively across all requirements.
- The AI feature is new to most team members. Someone needs to take ownership and research implementation before Phase II.
- Action: Each member to re-read the spec individually and come to the next meeting with a list of use cases they believe belong to their assigned area.

Meeting Memo #2 — Phase I Planning and Work Distribution

Date: 3/12/2026 at 1:00 PM

Location: Online (Zoom)

Attendees: Sangita Shrestha, Decong Deng, Gaurab Pun, Eddie Zhu, Tristan Chong

Discussion Summary:

- The team finalized the tech stack: Python (Flask) for the back-end, MySQL for the database, and HTML/CSS/JavaScript with Jinja2 templating for the front-end GUI. The vector store will use ChromaDB for the AI assistant feature, and the LLM integration will use the Claude, OpenAI, or Gemini API.
- Work was divided among team members. Front-end (GUI screens and navigation) was assigned to two members, back-end logic (user management, semester period enforcement, grading) to two members, and the AI feature and database schema to one member.
- The team reviewed all system requirements and identified the most complex areas: the semester period state machine, the review and warning system, and the grading period academic standing rules.
- The team outlined the major entities needed for the E-R diagram: User, Student, Instructor, Registrar, Course, Enrollment, Review, Warning, Complaint, Semester, and GraduationApplication.
- The git repository was created on GitHub. All members were added as collaborators. A branching strategy was agreed upon: each member works on a named feature branch and submits pull requests for review before merging to main.

Concerns / Action Items:

- Coordinating the front-end and back-end without stepping on each other's work requires clear API contracts. The team agreed to define a basic REST API structure before coding begins.
- The single-window GUI constraint (no pop-up windows) requires careful UI design planning. The front-end members will sketch wireframes before the next meeting.
- Back-end members to draft the database schema and share it to other members.

- Action: Front-end members to produce rough wireframes of the main screens (login, student dashboard, registrar dashboard, instructor dashboard).

Meeting Memo #3 — Design Review and Phase II Preparation

Date: 4/14/2026 at 3:30 PM

Location: On Campus Library

Attendees: Gaurab Pun, Eddie Zhu, Tristan Chong, Decong Deng, Sangita Shrestha

Discussion Summary:

- The team reviewed the draft database schema. The schema was debated, particularly around how to represent warnings (whether warnings should be a separate table linked to users, or stored as a counter attribute on the User entity). The team agreed on a separate Warnings table to preserve history and support the complaint/review/grading-triggered warning workflows.
- The collaboration class diagram was reviewed collectively. Relationships between User subclasses and Course, Enrollment, and Review entities were discussed and corrected. Multiplicity notations were verified against the requirements (e.g., a student can be enrolled in 2-4 courses; a course can have at most one instructor).
- All use cases were enumerated and documented. The team identified which use cases would use sequence/collaboration diagrams and which three would use Petri-nets. The Petri-net use cases selected were: (UC-06) course registration,

  (UC-08) course cancellation and special registration, and (UC-13) post-grading student status update, all of which involve concurrent state changes and are well-suited for Petri-net modeling.
- Pseudo-code was discussed for the most complex methods: registerCourse(), submitGrade(), processComplaint(), applyForGraduation(), and the AI query routing method queryAIAssistant(). Each of these was outlined at a high level to be expanded for the Phase II document.
- The creative feature was decided: the team will implement a real-time notification/alert panel for Registrars that surfaces urgent events such as instructor suspensions, student terminations, and courses at risk of cancellation due to low enrollment.

Concerns / Action Items:

- Writing pseudo-code for every method is very time-consuming. The team split the methods evenly: each member is responsible for the pseudo-code of the methods belonging to their assigned module.
- The Phase II deadline is tight. The team agreed to have all individual sections drafted before the deadline to allow time for review and integration into the final document.
- One team member raised the question of how to handle the special re-registration window during the class running period (when courses are cancelled). The team re-read the spec and

agreed this is a short, system-triggered second registration window that should be implemented as a sub-state within the class running period.

- Action: All members to complete their assigned pseudo-code sections.

7. Git Repository

All project materials including source code, design documents, meeting memos, diagrams, and this Phase II report are maintained in the team's GitHub repository. The repository is hosted under the personal GitHub account of one of the team members, with the remaining four team members added as collaborators with write access.

Repository URL: https://github.com/Gaurabpun/csc322-college0

Repository Structure:

- **main:** stable branch containing reviewed and finalized work.
- **design:** working branch for Phase II design documents, diagrams, and pseudo-code.
- Each team member works on a personal feature branch and submits pull requests to design before merging, ensuring all changes are peer-reviewed.

Contents of the Repository:

- **Phase I Report:** requirements analysis and initial use case documentation.
- **Phase II Report:** this design report, including the collaboration class diagram, use case diagrams, Petri-nets, E-R diagram, pseudo-code, system screen wireframes, and meeting memos.
- **Source code:** organized by module (frontend/, backend/, ai/, database/).
- **Diagrams:** all UML, E-R, and Petri-net diagrams exported as image files.
- **README.md:** project overview, setup instructions, and team information.

Note: The repository is public and accessible to the course instructor directly via the URL above.